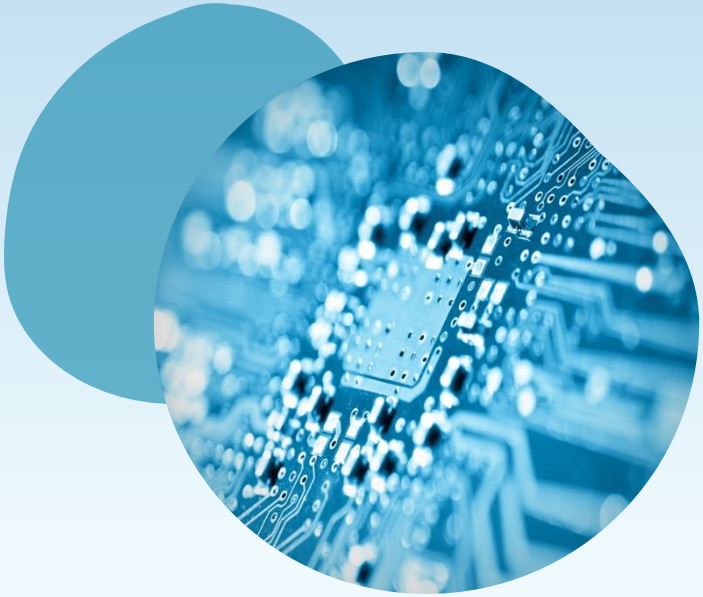


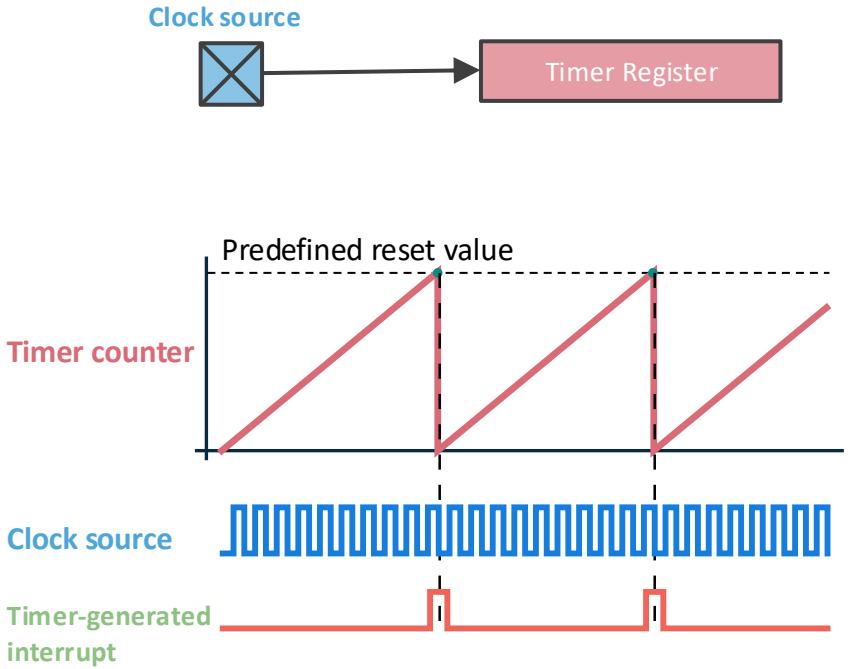
Timers & Clocks

Embedded Software for the Internet Of Things

Kasim Sinan Yildirim

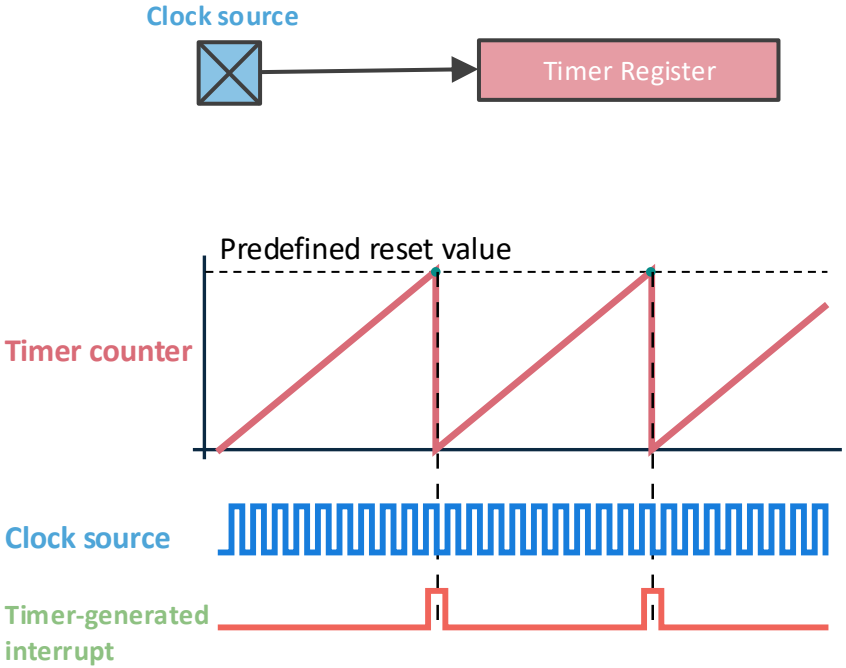
Timer Overview

- A **hardware timer** is a digital counter that:
  - Counts regular events, normally using a **clock source** with a **fixed frequency**
  - Increments or decrements at a fixed frequency
  - **Resets** itself when reaching zero, or a predefined value
  - Generates an **interrupt** when reset



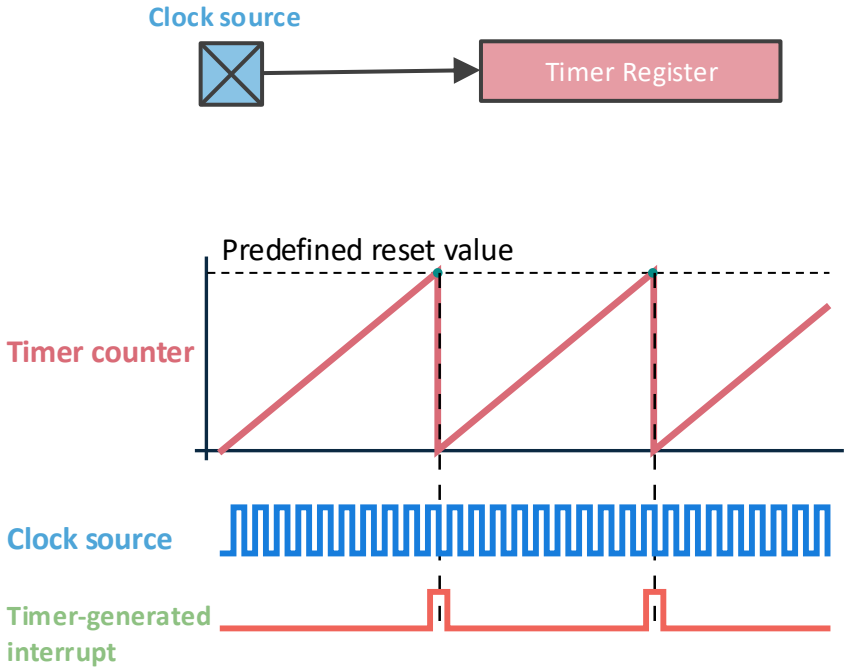
Timer Overview

- In contrast, a **software timer** is a similar function block but implemented in software.
  - A software timer is usually based on a hardware timer:
    - increments or decrements when interrupted
    - provides lower time precision compared with hardware timers
    - can have multiple instances, notably more than hardware timers



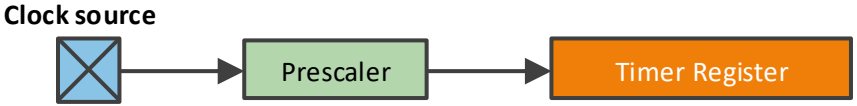
Timer Overview

- We use timers to:
  - Generate (periodic) events
  - Measure time passed to perform some computational tasks
  - Generate Pulse Width Modulation (PWM)
  - …



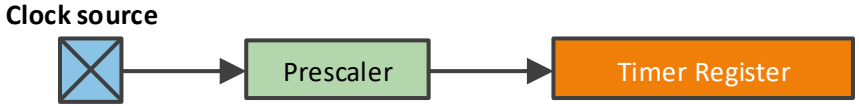
Components of a Standard Timer

- **Prescaler**
  - Takes the clock source as its input
  - Divides the input frequency by a predefined value, e.g., 4, 8, 16
  - Outputs the divided frequency to other components
- **Timer Register**
  - Is incremented or decremented at a fixed frequency
  - Is driven by the output from the prescaler, often referred to as “ticks”



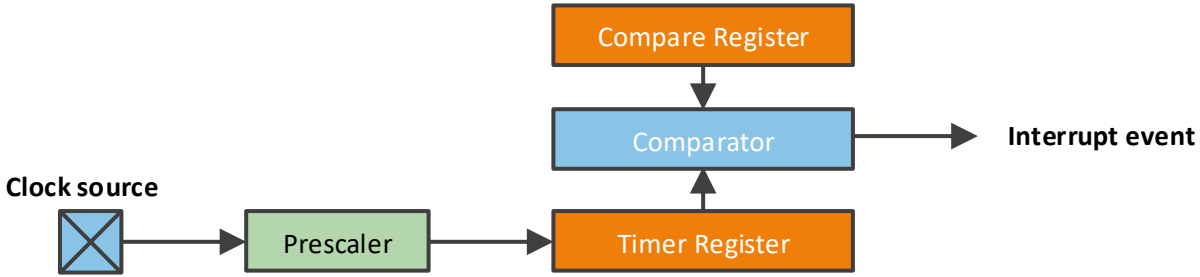
Timer Operation Mode

- Typically, a standard timer may have three operation modes:
  - Compare mode
  - Capture mode
  - PWM mode



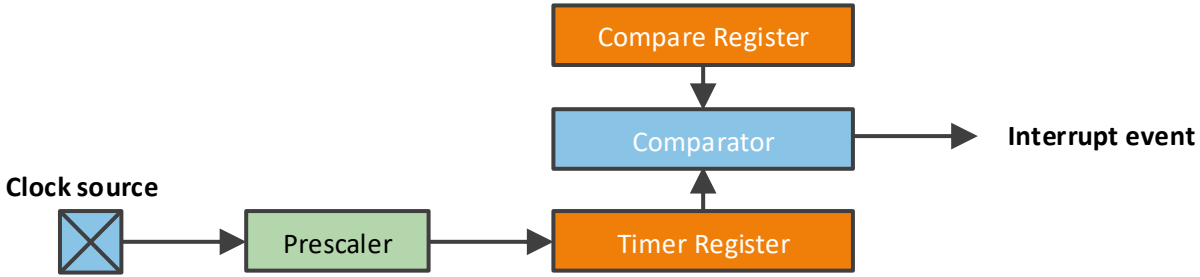
Timer Operation Mode

- **Compare Register**
  - Is loaded with a value, which is periodically compared with the value in the timer register
  - If the two values are the same, an interrupt can be generated



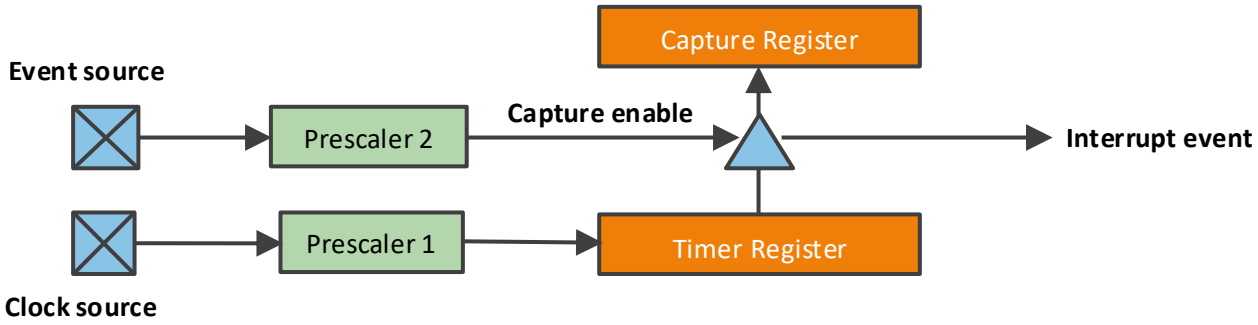
Timer Operation Mode

- **Compare Mode:**
  - Preload the compare register with a desired value
  - The values in the compare register and the timer register are automatically compared.
  - Once equal, an interrupt can be generated, and the timer register should be reset



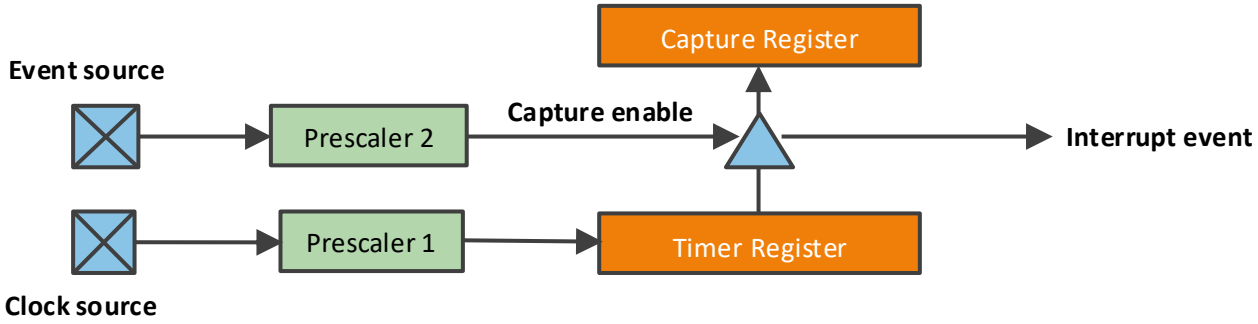
Timer Operation Mode

- **Capture Register**
  - Loads the current value from the timer register upon the occurrence of certain events
  - Can also generate an interrupt upon events



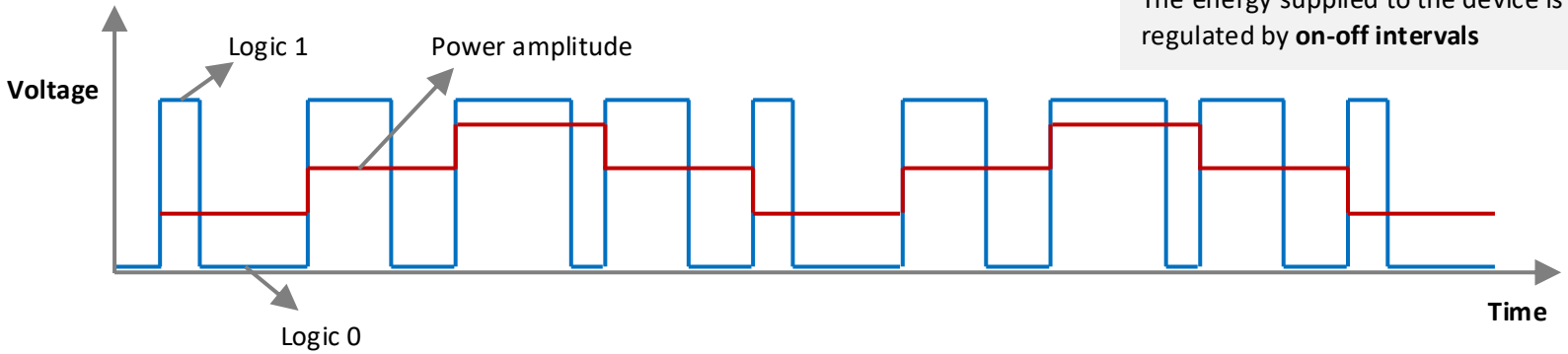
Timer Operation Mode

- **Capture Mode:**
  - The event source generates a sequence of pulses
    - Optionally, the prescaler can be used to divide the frequency of the events
  - The capture register then takes a '**snapshot**' of the timer register at the moment when the event occurs.



Timer Operation Mode

- Pulse-width modulation (**PWM**) Mode:
  - Uses the width of a pulse to modulate an amplitude
  - The amplitude reflects the duty cycle
    - which describes the proportion of the ‘1’ state in one pulse period
  - Mainly used to control the power supplied to electrical devices

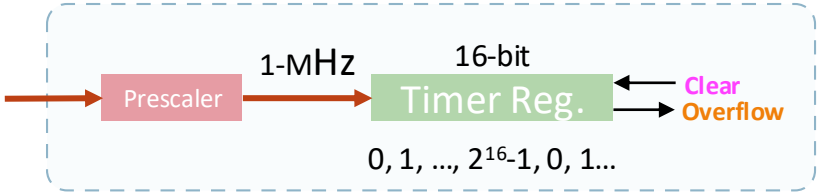


The energy supplied to the device is regulated by **on-off intervals**

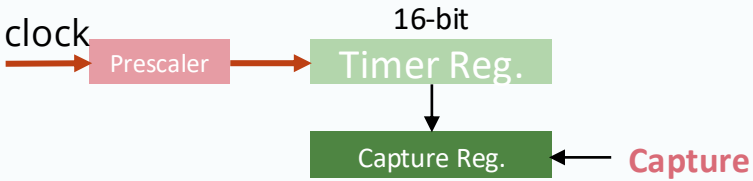
Summary: Timers

- Timers as Counters

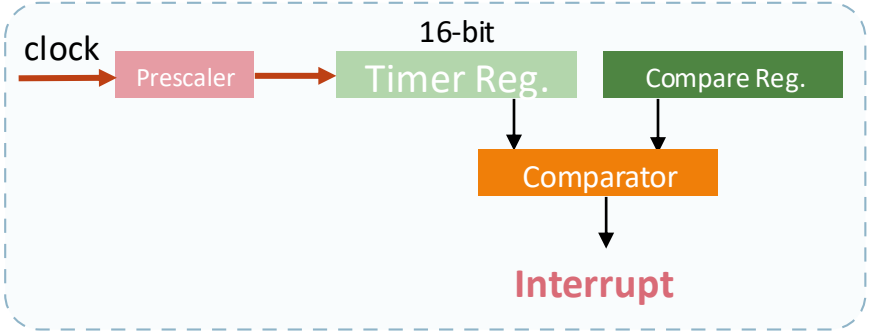




Capture Mode



Compare Mode



MSP432 Timer A

- TA0, TA1, TA2 and TA3
  - 16-bit timers

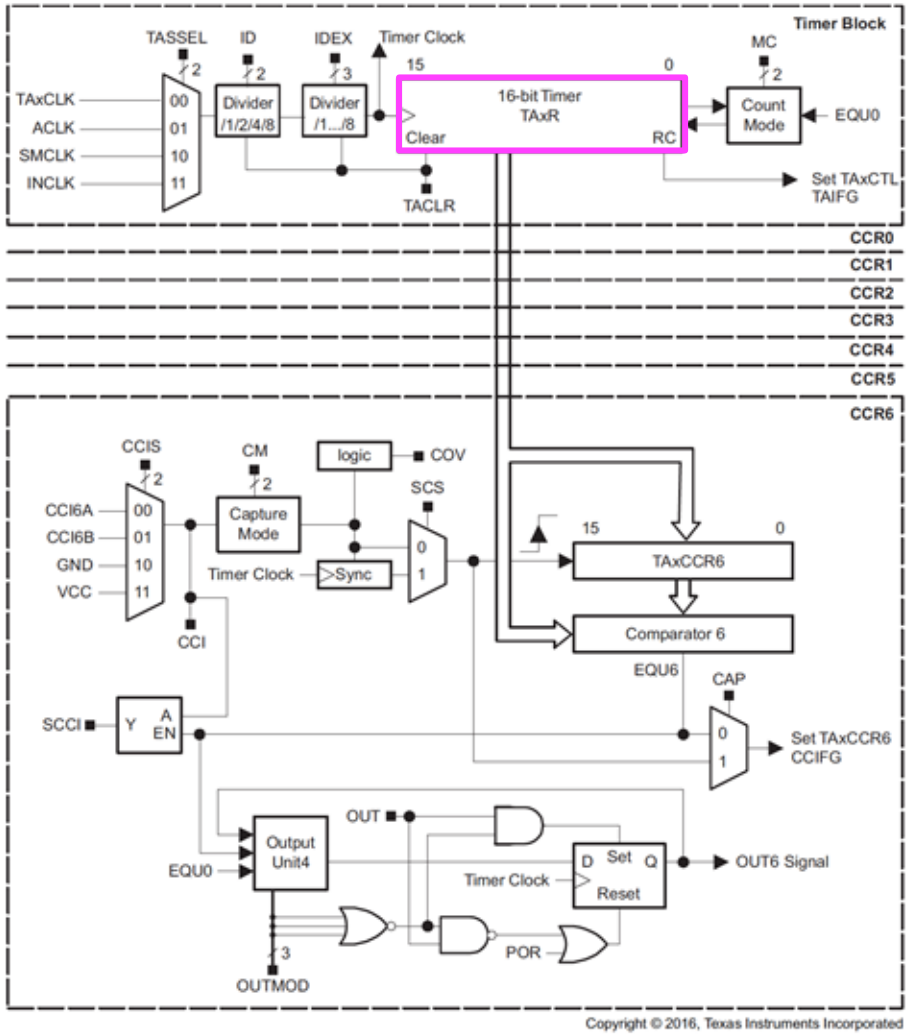


Figure 17-1. Timer_A Block Diagram

MSP432 Timer A

- TA0, TA1, TA2 and TA3
  - 16-bit timers
  - 7 capture/compare registers.
    - One physical timer but 7 blocks connected to it.

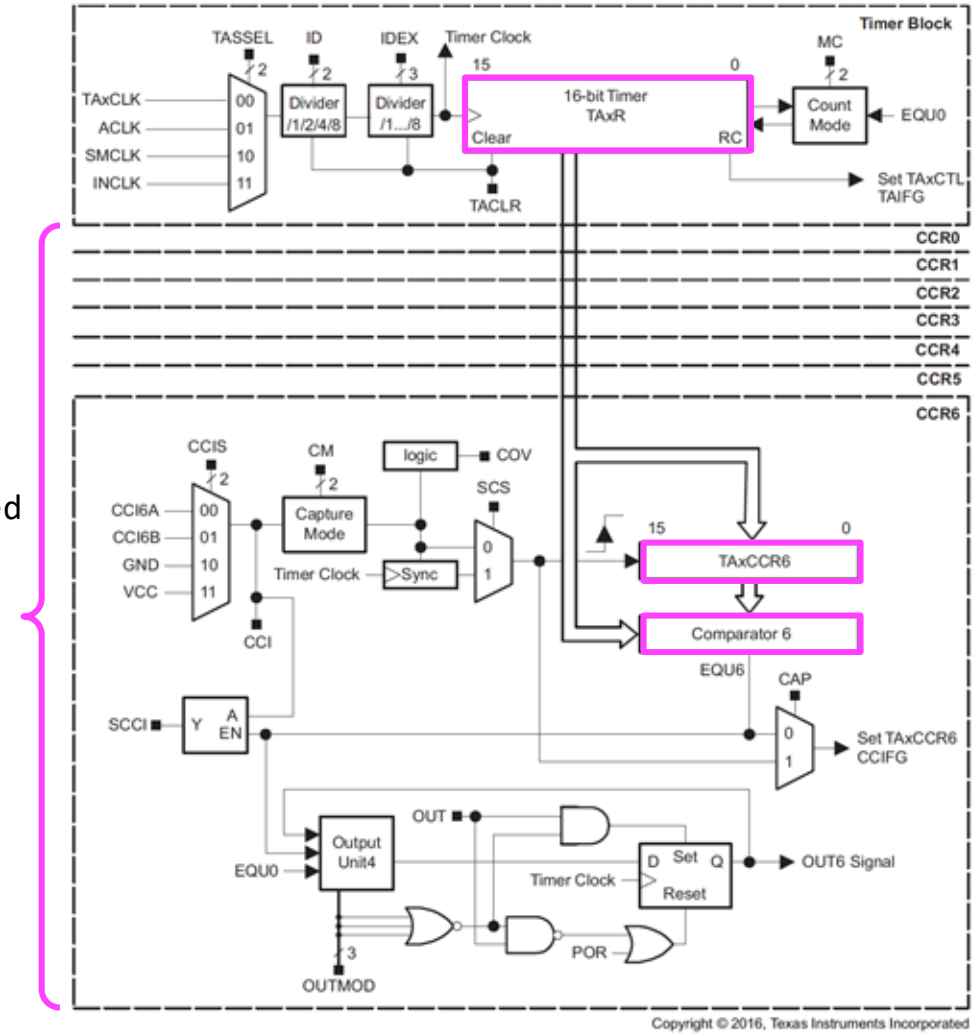
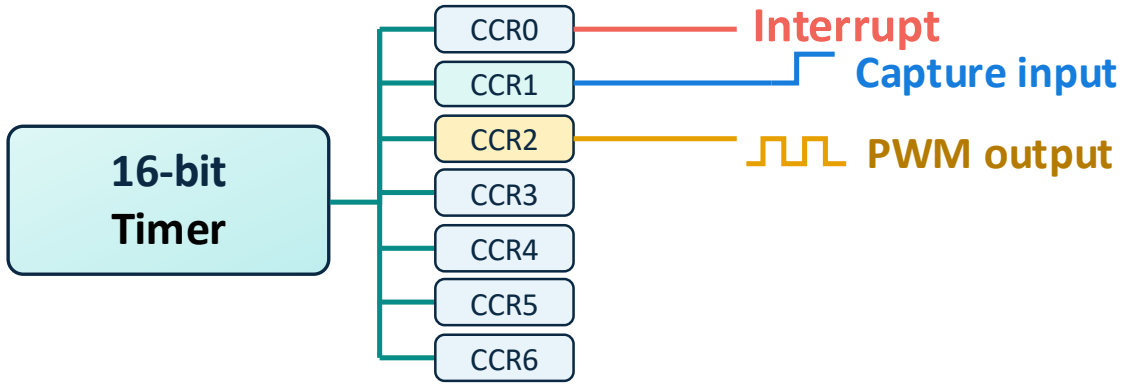
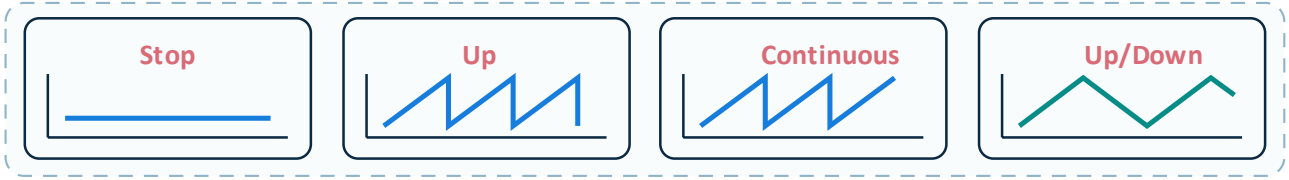


Figure 17-1. Timer_A Block Diagram

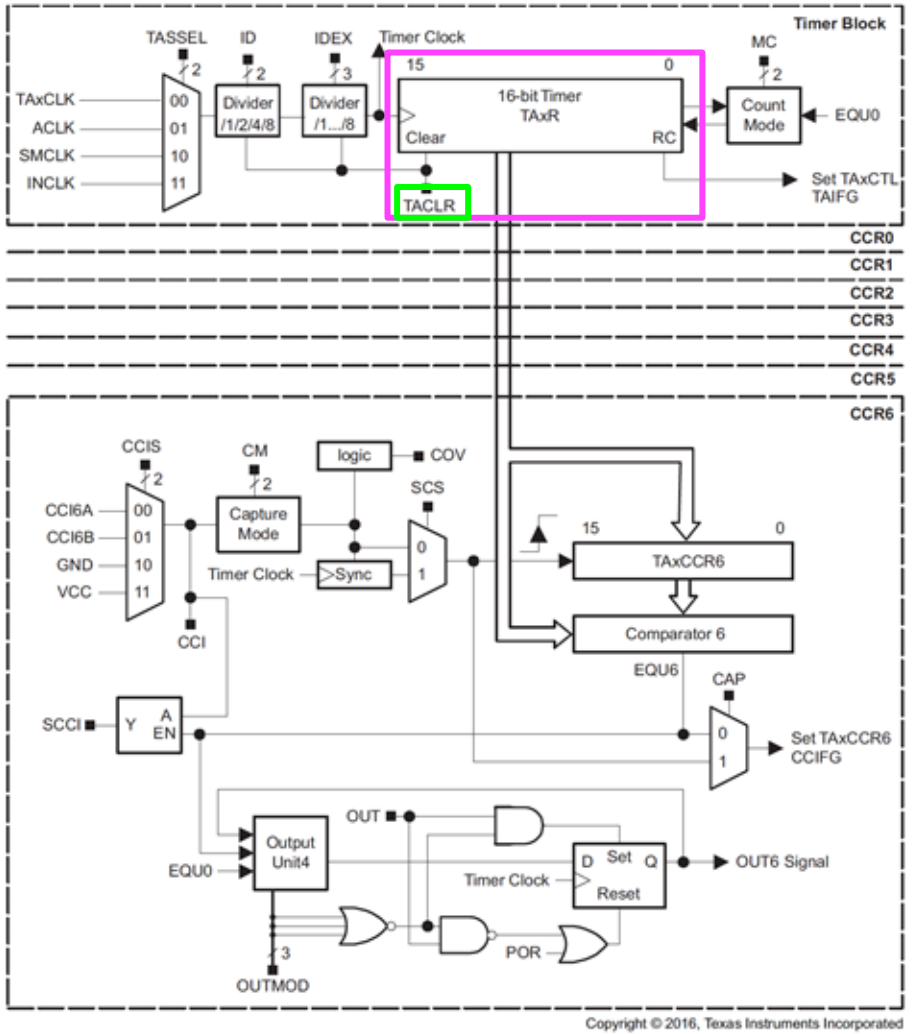
MSP432 Timer A





MSP432 Timer A

- **TAxR** 16-bit timer/counter register
  - increments or decrements (depending on mode of operation) with each rising edge of the clock signal.
  - Can be read or written with software.
    - cleared by setting the **TACLR** bit.



Copyright © 2016, Texas Instruments Incorporated

Figure 17-1. Timer_A Block Diagram

MSP432 Timer A

- The timer clock can be sourced from:
  - **ACLK**, **SMCLK**, or externally from **TAxCLK** or **INCLK**.
- The clock source is selected with the TASSEL bits.

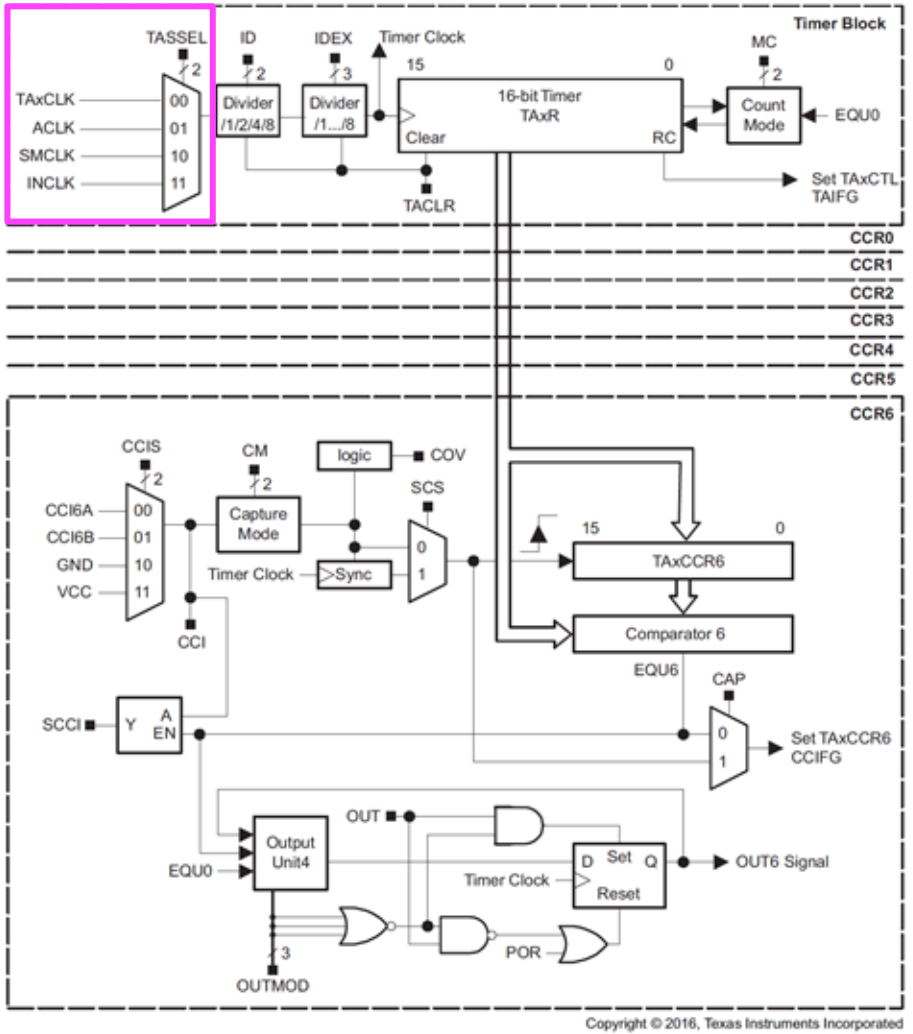
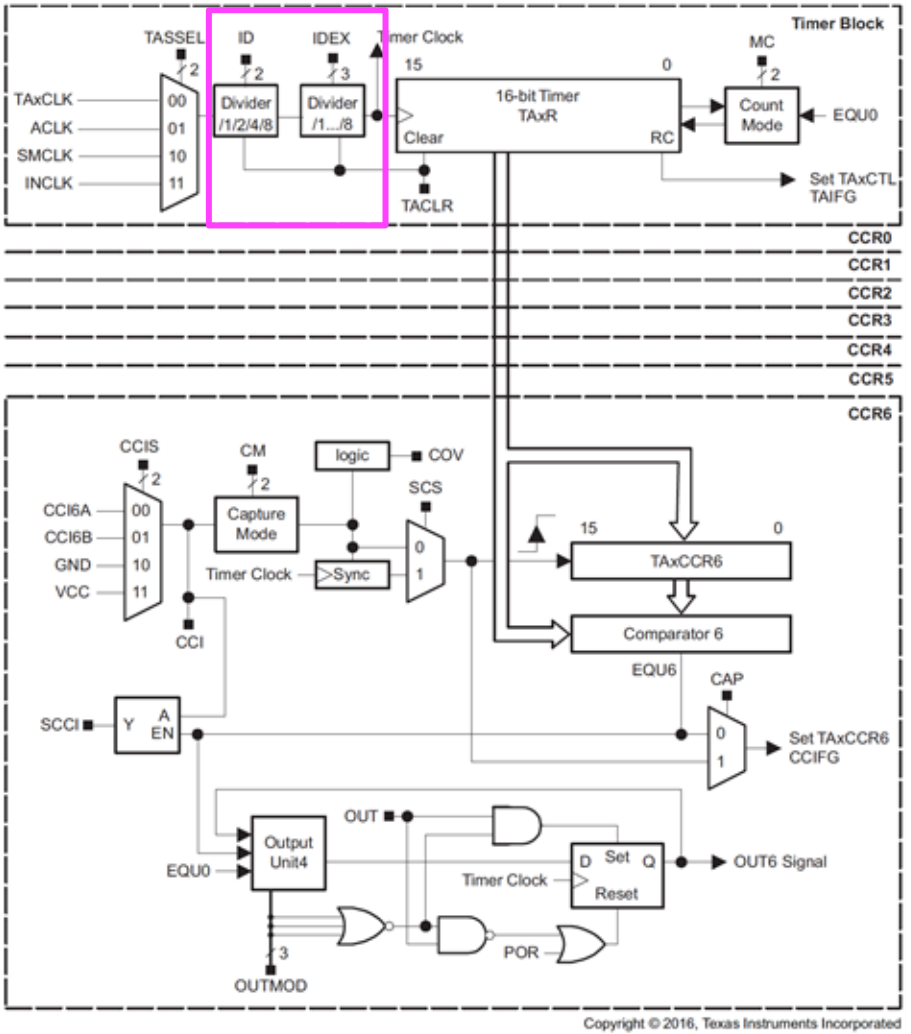


Figure 17-1. Timer_A Block Diagram

MSP432 Timer A

- The selected clock source may be passed directly to the timer
- Or divided by 2, 4, or 8, using the ID bits.
  - Can be further divided by 2, 3, 4, 5, 6, 7, or 8 using the TAIDEX bits.

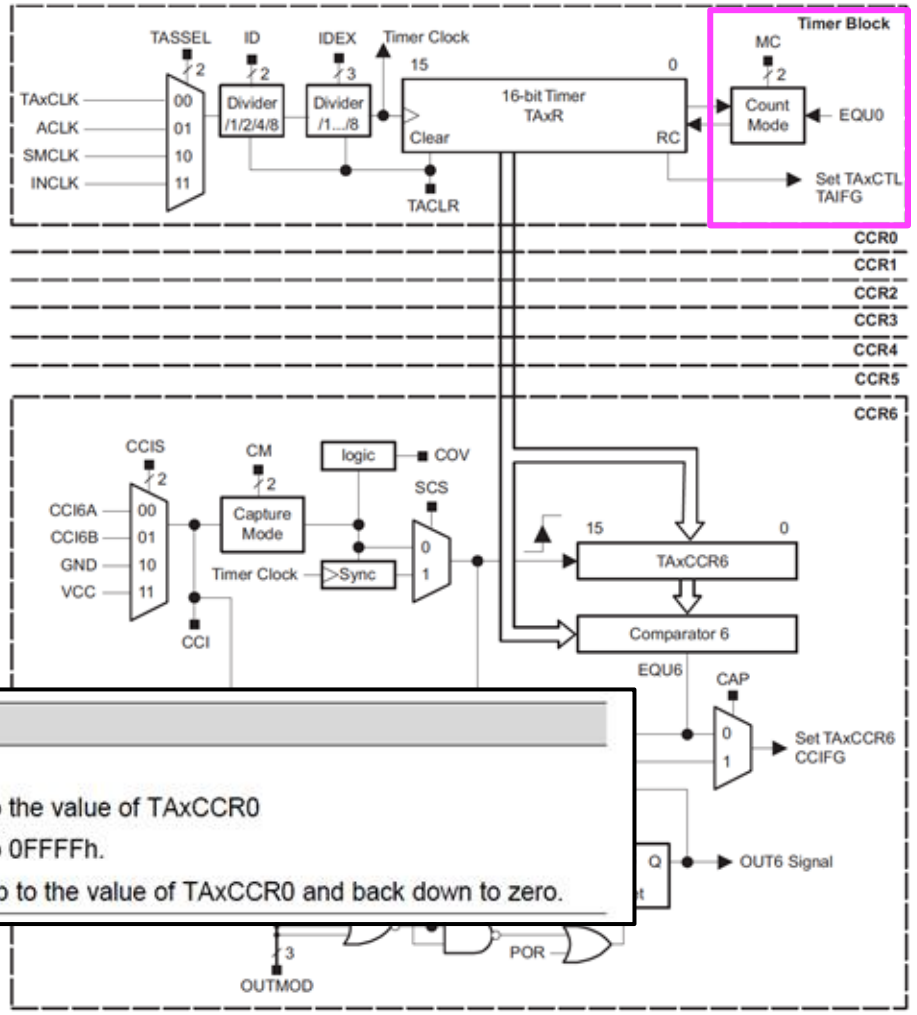


Copyright © 2016, Texas Instruments Incorporated

Figure 17-1. Timer_A Block Diagram

MSP432 Timer A

- The timer counts when **MC > 0** and the clock source is active.



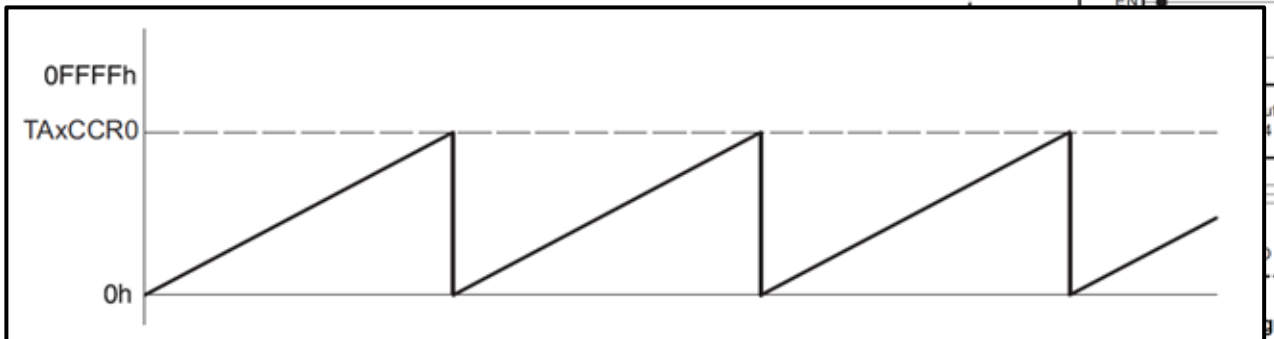
Copyright © 2016, Texas Instruments Incorporated

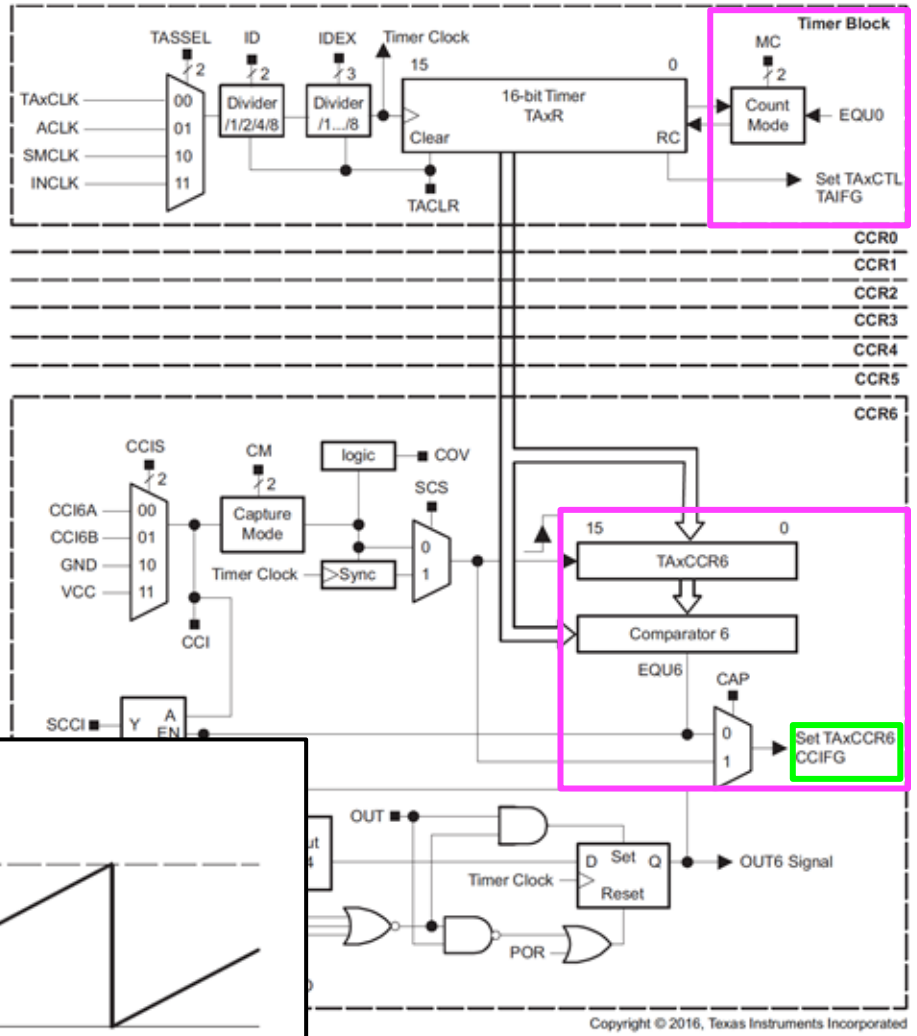
Figure 17-1. Timer_A Block Diagram

MC	Mode	Description
00	Stop	The timer is halted.
01	Up	The timer repeatedly counts from zero to the value of TAxCCR0
10	Continuous	The timer repeatedly counts from zero to 0FFFFh.
11	Up/down	The timer repeatedly counts from zero up to the value of TAxCCR0 and back down to zero.

MSP432 Timer A

- Example: Up Mode
  - The timer repeatedly counts up to the value of compare register **TAxCCR0**
  - The **TAxCCR0 CCIFG** interrupt flag is set
    - (to receive TACCR0 interrupt, we need to enable it)
  - Then, the timer restarts counting from zero.





gure 17-1. Timer_A Block Diagram

MSP432 Timer A (Example)

- TAxCCTL0 to TAxCCTL6 Capture/ Compare Control Register

17.3.3 TAxCCTL0 to TAxCCTL6 Register

Timer_Ax Capture/Compare Control 0 Register to Timer_Ax Capture/Compare Control 6

Figure 17-17. TAxCCTL0 to TAxCCTL6 Register

15	14	13	12	11	10	9	8
CM		CCIS		SCS	SCCI	Reserved	CAP
rw-0	rw-0	rw-0	rw-0	rw-0	r-0	r-0	rw-0
7	6	5	4	3	2	1	0
OUTMOD			CCIE	CCI	OUT	COV	CCIFG
rw-0	rw-0	rw-0	rw-0	r	rw-0	rw-0	rw-0

bit	Name	Description
15-14	CM	Capture mode 00: No capture 01: Capture on rising edge 10: Capture on falling edge 11: Capture on both rising and falling edges
13-12	CCIS	Capture/compare input select the TAxCCR0 input signal: 00: CCIxA 01: CCIxB 10: GND 11: VCC
11	SCS	Synchronize capture source. This bit is used to synchronize the capture input signal with the timer clock. 0: Asynchronous capture 1: Synchronous capture
10	SCCI	Synchronized capture/compare input. The selected CCI input signal is latched with the EQUx signal and can be read via this bit.
8	CAP	Capture mode 0: Compare mode 1: Capture mode
7-5	OUTMOD	Output mode. Modes 2, 3, 6, and 7 are not useful for TAxCCR0 because EQUx = EQU0 000: OUT bit value 001: Set 010: Toggle/reset 011: Set/reset 100: Toggle 101: Reset 110: Toggle/set 111: Reset/set
4	CCIE	Capture/compare interrupt enable. This bit enables the interrupt request of the corresponding CCIFG flag. 0: Interrupt disabled 1: Interrupt enabled
3	CCI	Capture/compare input. The selected input signal can be read by this bit.
2	OUT	Output. For output mode 0, this bit directly controls the state of the output. 0: Output low 1: Output high
1	COV	Capture overflow. This bit indicates a capture overflow occurred. COV must be reset with software. 0: No capture overflow occurred 1: Capture overflow occurred
0	CCIFG	Capture/compare interrupt flag 0: No interrupt pending 1: interrupt pending

MSP432 Timer A (Example)

- Set the CCIE bit in TA0 CCTL0 register to enable interrupt

```
TIMER_A0->CCTL[0] =TIMER_A_CCTLN_CCIE;
```

bit	Name	Description
15-14	CM	Capture mode 00: No capture 01: Capture on rising edge 10: Capture on falling edge 11: Capture on both rising and falling edges
13-12	CCIS	Capture/compare input select the TAxCCR0 input signal: 00: CCIxA 01: CCIxB 10: GND 11: VCC
11	SCS	Synchronize capture source. This bit is used to synchronize the capture input signal with the timer clock. 0: Asynchronous capture 1: Synchronous capture
10	SCCI	Synchronized capture/compare input. The selected CCI input signal is latched with the EQUx signal and can be read via this bit.
8	CAP	Capture mode 0: Compare mode 1: Capture mode
7-5	OUTMOD	Output mode. Modes 2, 3, 6, and 7 are not useful for TAxCCR0 because EQUx = EQU0 000: OUT bit value 001: Set 010: Toggle/reset 011: Set/reset 100: Toggle 101: Reset 110: Toggle/set 111: Reset/set
4	CCIE	Capture/compare interrupt enable. This bit enables the interrupt request of the corresponding CCIFG flag. 0: Interrupt disabled 1: Interrupt enabled
3	CCI	Capture/compare input. The selected input signal can be read by this bit.
2	OUT	Output. For output mode 0, this bit directly controls the state of the output. 0: Output low 1: Output high
1	COV	Capture overflow. This bit indicates a capture overflow occurred. COV must be reset with software. 0: No capture overflow occurred 1: Capture overflow occurred
0	CCIFG	Capture/compare interrupt flag 0: No interrupt pending 1: interrupt pending

MSP432 Timer A (Example)

- Set the value for the comparison.
  - The timer will count up to 50000.

```
TIMER_A0->CCTL[0] =TIMER_A_CCTLN_CCIE;
TIMER_A0->CCR[0] = 50000;
```

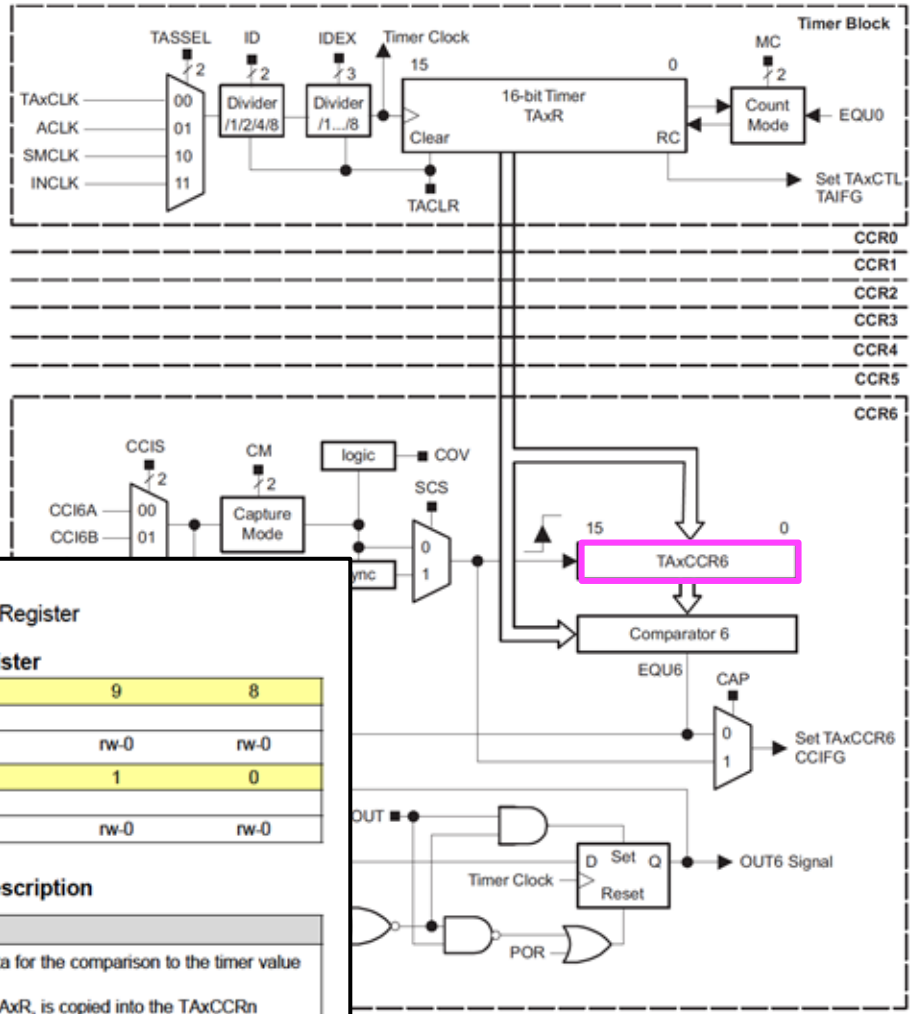


Figure 17-1. Timer_A Block Diagram

17.3.4 TAxCCR0 to TAxCCR6 Register

Timer_Ax Capture/Compare 0 Register to Timer_Ax Capture/Compare 6 Register

Figure 17-18. TAxCCR0 to TAxCCR6 Register

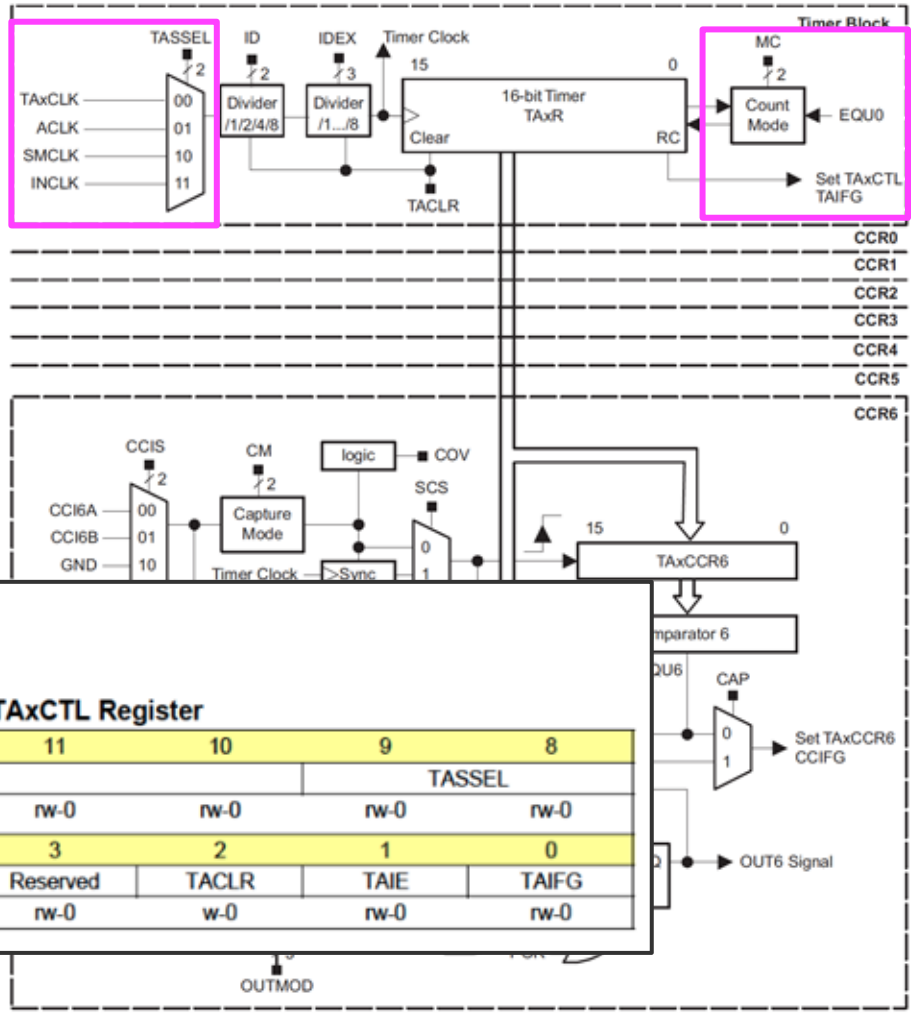
15	14	13	12	11	10	9	8
TAxCCRn							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0
7	6	5	4	3	2	1	0
TAxCCRn							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 17-7. TAxCCR0 to TAxCCR6 Register Description

Bit	Field	Type	Reset	Description
15-0	TAxCCR0	RW	0h	Compare mode: TAxCCRn holds the data for the comparison to the timer value in the Timer_A Register, TAxR. Capture mode: The Timer_A Register, TAxR, is copied into the TAxCCRn register when a capture is performed.

MSP432 Timer A (Example)

- Set clock source as SMCLK (**3 MHz**) and continuous mode.
  - Use TA0CTL Control Register for this configuration.



Copyright © 2016, Texas Instruments Incorporated

Figure 17-1. Timer_A Block Diagram

17.3.1 TAxCTL Register

Timer_Ax Control Register

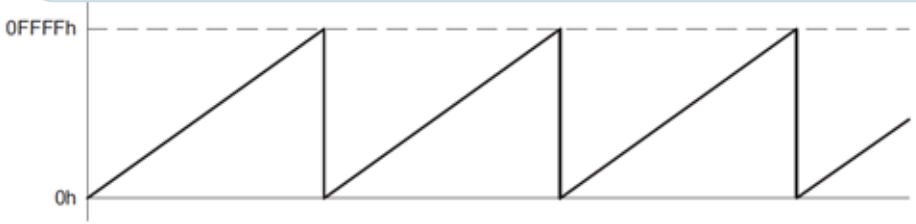
Figure 17-15. TAxCTL Register

15	14	13	12	11	10	9	8
Reserved						TASSEL	
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0
7	6	5	4	3	2	1	0
ID		MC		Reserved	TACLR	TAIE	TAIFG
rw-0	rw-0	rw-0	rw-0	rw-0	w-0	rw-0	rw-0

MSP432 Timer A (Example)

- Set clock source as SMCLK (3 MHz) and continuous mode.
  - Use TA0CTL Control Register for this configuration.

```
TIMER_A0->CCTL[0] =TIMER_A_CCTLN_CCIE;
TIMER_A0->CCR[0] = 50000;
TIMER_A0->CTL = TIMER_A_CTL_SSEL__SMCLK|
            TIMER_A_CTL_MC__CONTINUOUS;
```

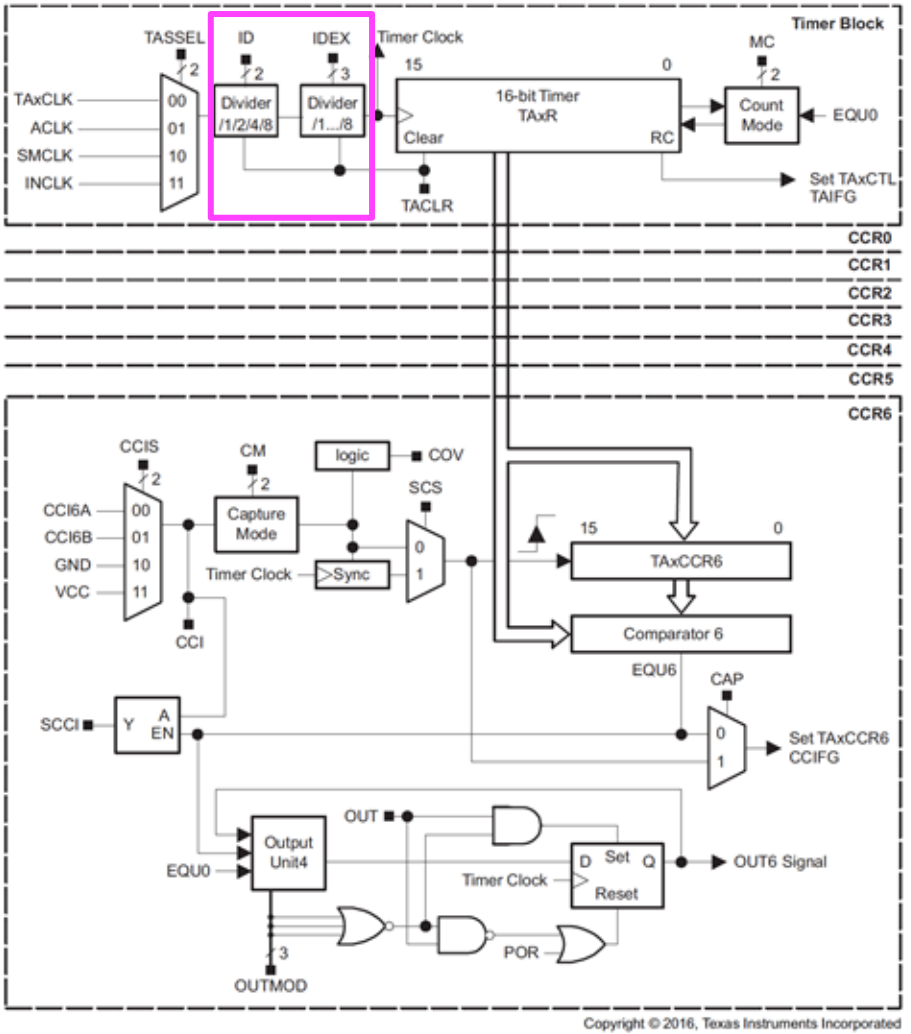


TAxCTL Register

bit	Name	Description
0	TAIFG	Timer_A Interrupt Flag 0: Timer did not overflow 1: Timer overflowed
1	TAIE	Timer_A Interrupt Enable (0: Disabled, 1: Enabled)
2	TACLR	Timer_A Clear
4-5	MC	Mode Control: 00: Stop mode: timer is halted 01: Up mode: Timer counts up to TAxCCR0 10: Continuous mode: Timer counts up to 0xFFFF 11: Up/down mode: Timer counts up to TAxCCR0 then down to 0.
6-7	ID	Input divider: These bits select the divider for the input clock: 00: divide by 1 01: divide by 2 10: divide by 4 11: divide by 8
8-9	TASSEL	Timer_A clock Source Select: These bits select the Timer_A clock source: 00: TAxCLK (external clock): The timer uses external clock which is fed to the PM_TAxCLK pin. 01: ACLK (internal clock) 10: SMCLK (internal clock) 11: INCLK

MSP432 Timer A (Example)

- You can also use an input divider for SMCLK to slow down the input clock signal (**PRESCALER**).
  - E.g., use divider 8
- By default **SMCLK = 3 MHz**
  - 3000000 ticks per second
    - divider 8 -> 3 Mhz / 8 => 375000 ticks per second
    - 375000/(2^16-1) -> almost 6 interrupts per second



Copyright © 2016, Texas Instruments Incorporated

Figure 17-1. Timer_A Block Diagram

MSP432 Timer A (Example)

- Input divider 8 for SMCLK to slow down the input clock signal

```
TIMER_A0->CCTL[0] =TIMER_A_CCTLN_CCIE;
TIMER_A0->CCR[0] = 50000;
TIMER_A0->CTL = TIMER_A_CTL_SSEL__SMCLK|
          TIMER_A_CTL_MC__CONTINUOUS|
                TIMER_A_CTL_ID_3;
```

TAxCTL Register

bit	Name	Description
0	TAIFG	Timer_A Interrupt Flag 0: Timer did not overflow 1: Timer overflowed
1	TAIE	Timer_A Interrupt Enable (0: Disabled, 1: Enabled)
2	TACLR	Timer_A Clear
4-5	MC	Mode Control: 00: Stop mode: timer is halted 01: Up mode: Timer counts up to TAxCCR0 10: Continuous mode: Timer counts up to 0xFFFF 11: Up/down mode: Timer counts up to TAxCCR0 then down to 0.
6-7	ID	**Input divider: These bits select the divider for the input clock: 00: divide by 1 01: divide by 2 10: divide by 4 11: divide by 8**
8-9	TASSEL	Timer_A clock Source Select: These bits select the Timer_A clock source: 00: TAxCLK (external clock): The timer uses external clock which is fed to the PM_TAxCLK pin. 01: ACLK (internal clock) 10: SMCLK (internal clock) 11: INCLK

MSP432 Timer A (Example)

- Enable timer **IRQ**
- Two interrupt vectors are associated with the 16-bit Timer_A module:
  - **TAxCCR0 interrupt vector** for TAxCCR0 CCIFG (Capture Compare Interrupt Flag)
  - **TAxIV interrupt vector** for all other CCIFG flags and TAIFG (TimerA Interrupt Flag)

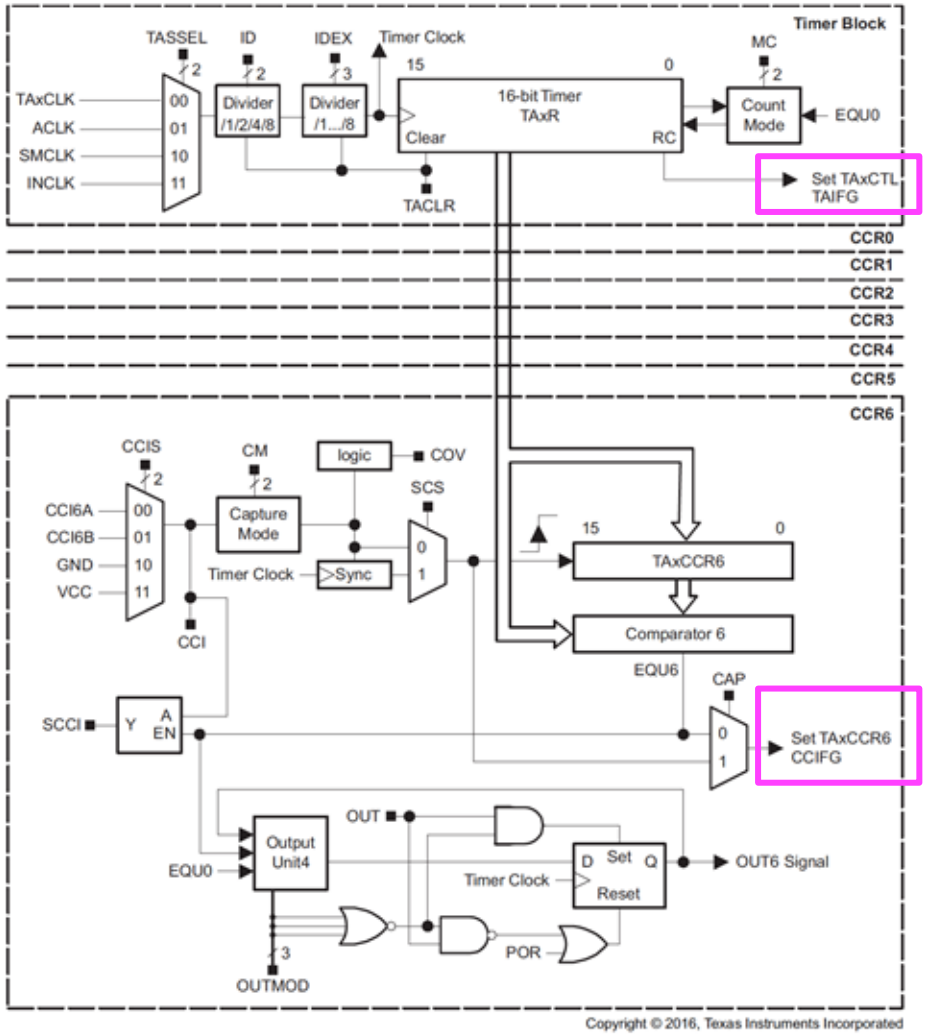


Figure 17-1. Timer_A Block Diagram

MSP432 Timer A (Example)

- Enable timer IRQ

We have a **specific IRQ line** for CCR0, others share the same IRQ.

For TA0CCR0 CCIFG interrupt

For other TA0CCR1–6 CCIFG and TA0IFG interrupts

INT#	IRQ#	Vector location	Device
1-15	None	0000 0000 to 0000 003C	CPU Exception(set by ARM)
16	0	0000 0040	PSS
17	1	0000 0044	CS
18	2	0000 0048	PCM
19	3	0000 004C	WDT_A
20	4	0000 0050	FPU_INT
21	5	0000 0054	Flash Controller
22	6	0000 0058	COMP_E1
23	7	0000 005C	COMP_E2
24	8	0000 0060	TIMERA0
25	9	0000 0064	TIMERA0
26	10	0000 0068	TIMERA1
27	11	0000 006C	TIMERA1
28	12	0000 0070	TIMERA2

MSP432 Timer A (Example)

- Enable timer IRQ

```
TIMER_A0->CCTL[0] =TIMER_A_CCTLN_CCIE;
TIMER_A0->CCR[0] = 50000;
TIMER_A0->CTL = TIMER_A_CTL_SSEL__SMCLK|
           TIMER_A_CTL_MC__CONTINUOUS|
                   TIMER_A_CTL_ID_3;

NVIC->ISER[0] = 1 << ((TA0_0_IRQn) & 31);
```

MSP432 Timer A (Example)

- Timer Interrupt Handler

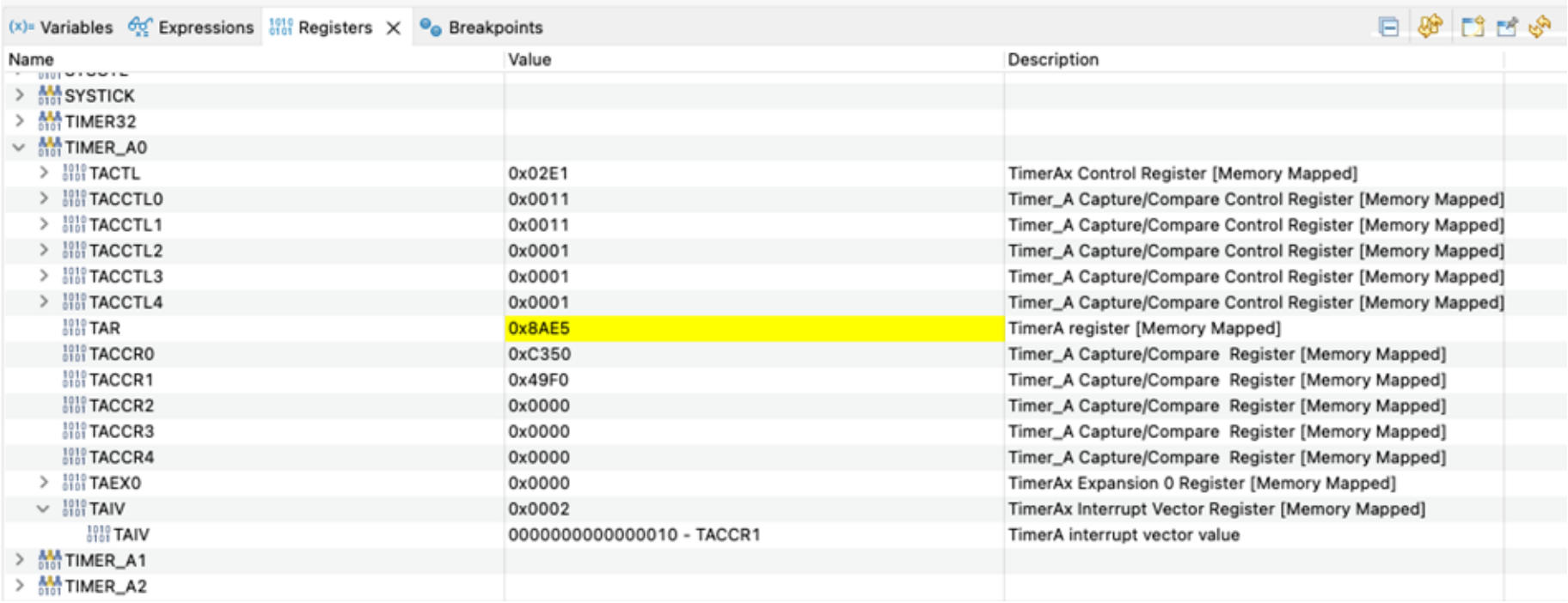
```
// will be called when TA0CCR0 CCIFG is set
void TA0_0_IRQHandler(){

    // clear the interrupt flag
    TIMER_A0->CCTL[0] &= ~TIMER_A_CCTLN_CCIFG;

    // toggle LED
    P1->OUT ^= BIT0;
}
```

MSP432 Timer A (Example)

- Explore TIMER_A0 Registers



Name	Value	Description
SYSTICK		
TIMER32		
TIMER_A0		
TACTL	0x02E1	TimerAx Control Register [Memory Mapped]
TACCTL0	0x0011	Timer_A Capture/Compare Control Register [Memory Mapped]
TACCTL1	0x0011	Timer_A Capture/Compare Control Register [Memory Mapped]
TACCTL2	0x0001	Timer_A Capture/Compare Control Register [Memory Mapped]
TACCTL3	0x0001	Timer_A Capture/Compare Control Register [Memory Mapped]
TACCTL4	0x0001	Timer_A Capture/Compare Control Register [Memory Mapped]
TAR	0x8AE5	TimerA register [Memory Mapped]
TACCR0	0xC350	Timer_A Capture/Compare Register [Memory Mapped]
TACCR1	0x49F0	Timer_A Capture/Compare Register [Memory Mapped]
TACCR2	0x0000	Timer_A Capture/Compare Register [Memory Mapped]
TACCR3	0x0000	Timer_A Capture/Compare Register [Memory Mapped]
TACCR4	0x0000	Timer_A Capture/Compare Register [Memory Mapped]
TAEX0	0x0000	TimerAx Expansion 0 Register [Memory Mapped]
TAIV	0x0002	TimerAx Interrupt Vector Register [Memory Mapped]
TAIV	0000000000000010 - TACCR1	TimerA interrupt vector value
TIMER_A1		
TIMER_A2		

Lab#0

- Select SMCKL (3 MHz) as clock source to Timer A0 and create a **1 second** (approximately) timer that toggles blue LED.

Lab#1

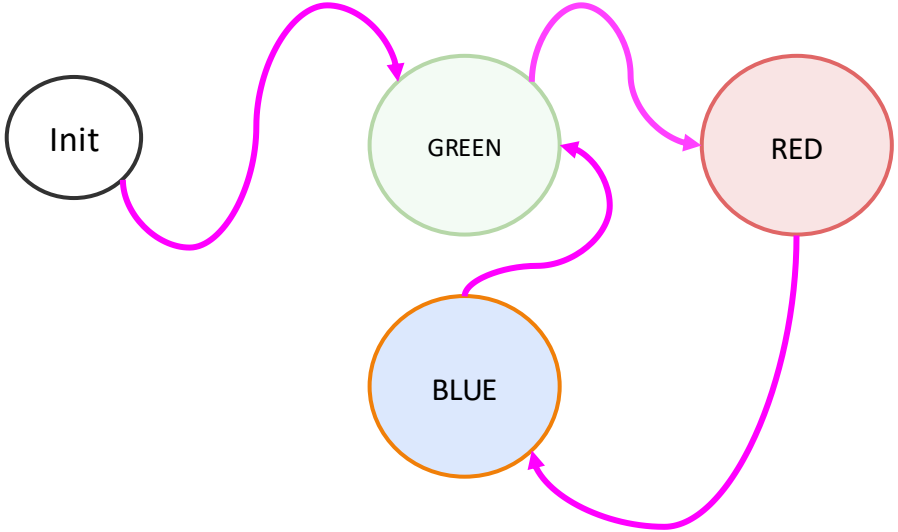
- User Timer A0:
  - Use CCR0 to toggle the Red LED with every second.
  - Use CCR1 to toggle the Green LED with every 3 seconds.
    - You need to use TA0_N_IRQHandler and enable TA0_N_IRQn IRQ line to capture interrupts for CCR1 -> Check TAIV bits.

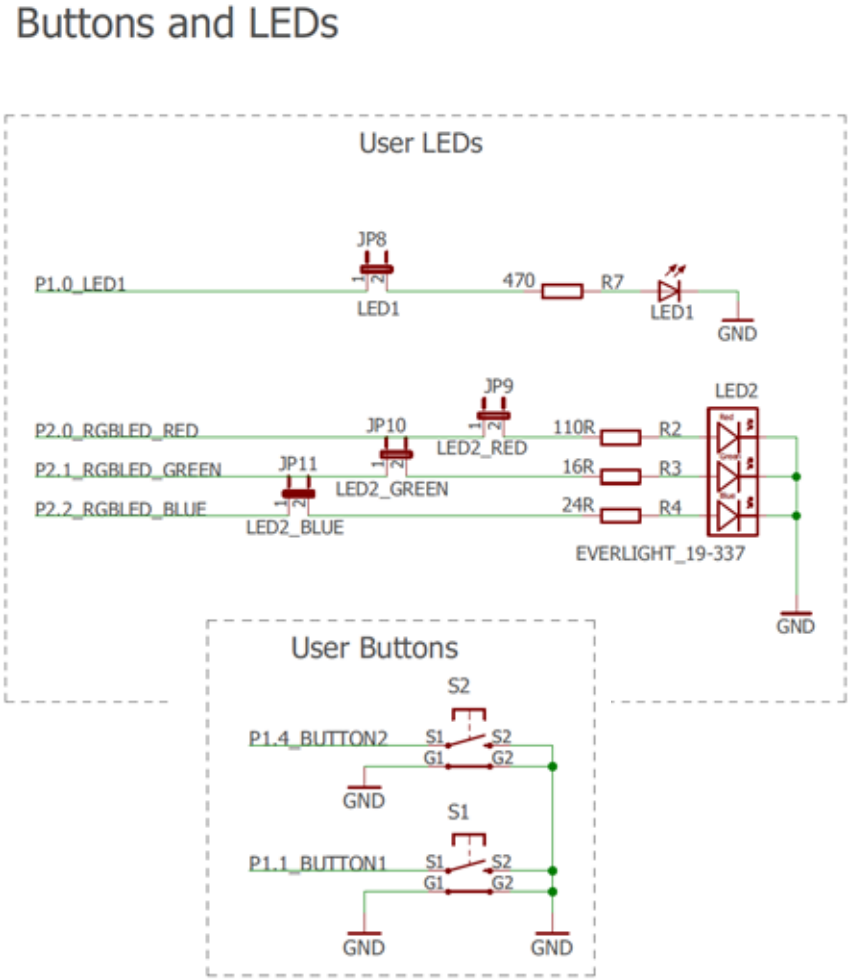
Bit	Field	Type	Reset	Description
15-0	TAIV	R	0h	Timer_A interrupt vector value 00h = No interrupt pending 02h = Interrupt Source: Capture/compare 1; Interrupt Flag: TAxCCR1 CCIFG; Interrupt Priority: Highest 04h = Interrupt Source: Capture/compare 2; Interrupt Flag: TAxCCR2 CCIFG 06h = Interrupt Source: Capture/compare 3; Interrupt Flag: TAxCCR3 CCIFG 08h = Interrupt Source: Capture/compare 4; Interrupt Flag: TAxCCR4 CCIFG 0Ah = Interrupt Source: Capture/compare 5; Interrupt Flag: TAxCCR5 CCIFG 0Ch = Interrupt Source: Capture/compare 6; Interrupt Flag: TAxCCR6 CCIFG 0Eh = Interrupt Source: Timer overflow; Interrupt Flag: TAxCTL TAIFG; Interrupt Priority: Lowest

Also catch **timer overflow** interrupt!

Lab #2

- Implement the following FSM with with timers.
  - Each state transition will take place **every 1 second**.





Watchdog Timer

- Watchdog module performs a controlled system restart after a software problem occurs.
- Watchdog timer counts down from a selected interval.
- If the selected time interval expires, a system reset is generated.

Watchdog Timer

- WDTCTL
  - 16-bit password-protected read/write register.
  - Write accesses must include the write password 05Ah in the upper byte.
  - Setting WDTHOLD = 1 stops watchdog timer.

```
WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD;
```

password 05Ah

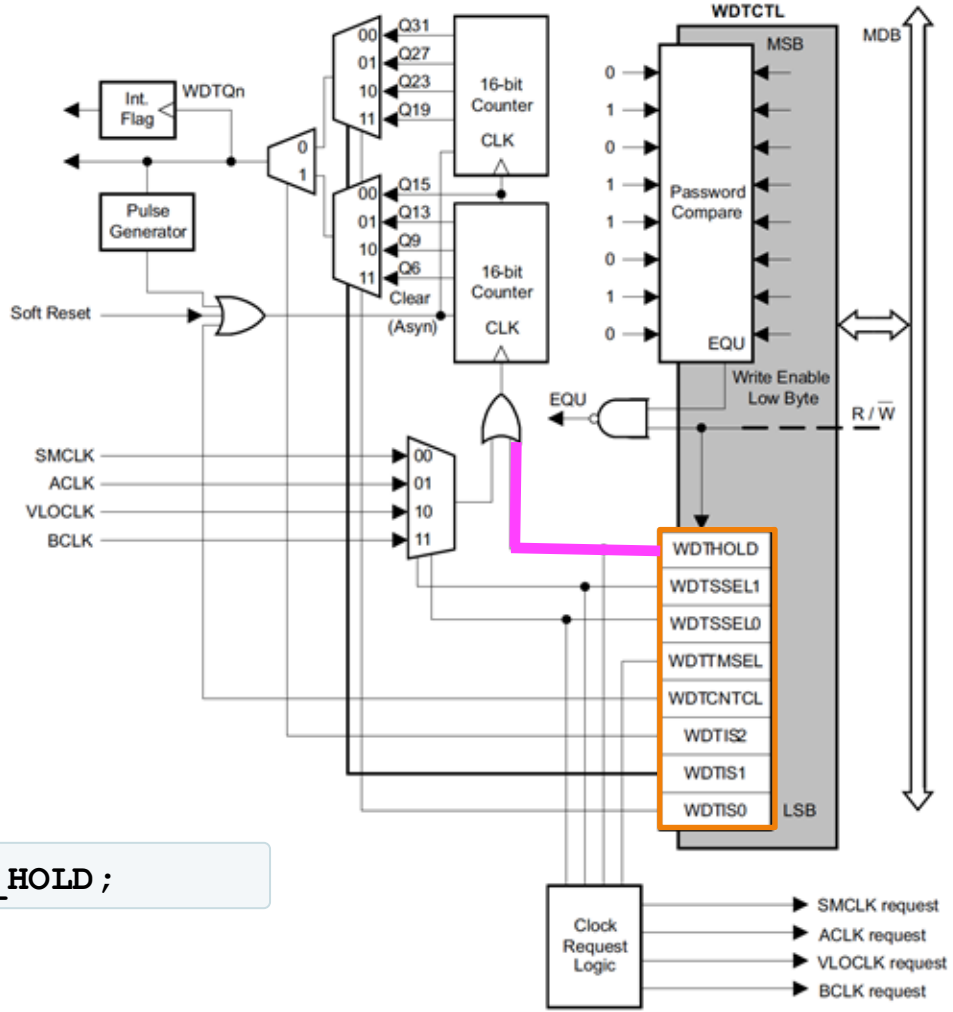
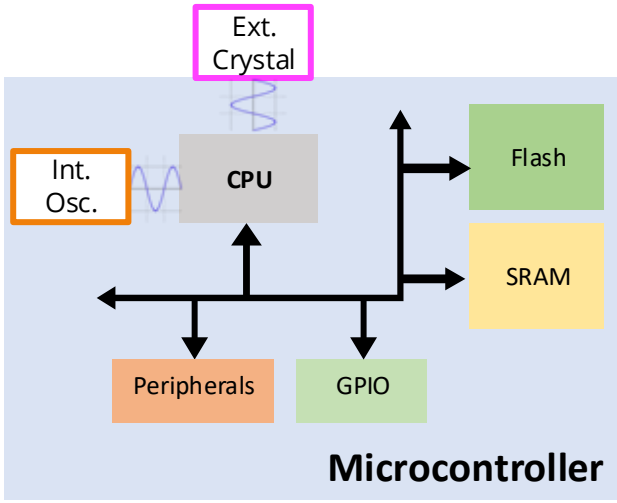


Figure 15-1. Watchdog Timer Block Diagram

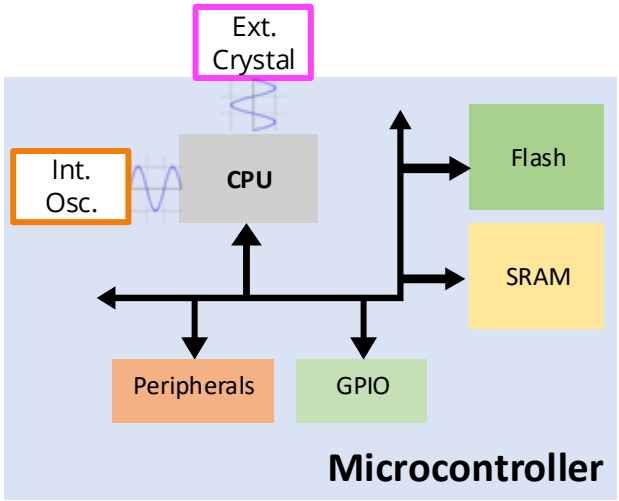
Clocks in MSP432

- Usually microcontrollers have at least two types of clock source:
  - the **on-chip oscillator** and the oscillator connected to **external crystal**
- The advantage of the external crystal oscillator is the higher precision
- The advantage of the on-chip oscillator is that it is always there



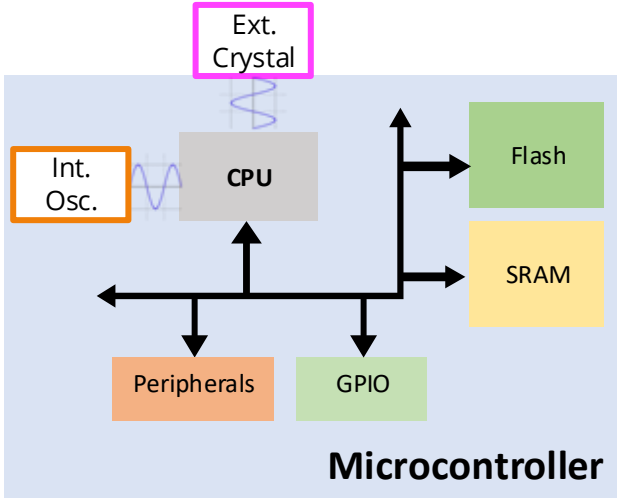
Clocks in MSP432

- The clock system contains the sources of the various clocks in the device.
- It also controls the mapping between the sources and the different clocks in the device.



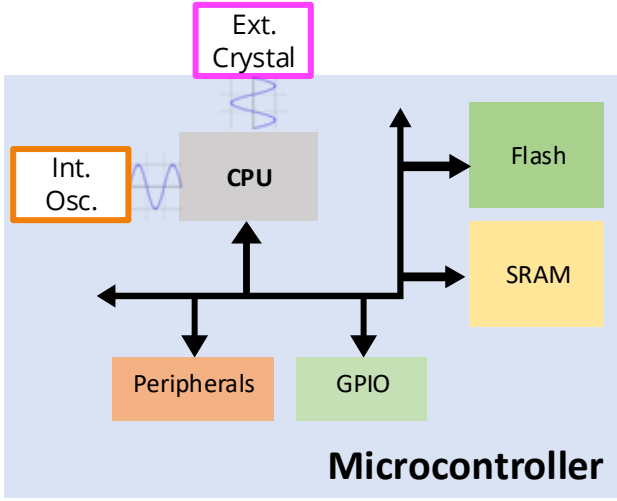
Clocks in MSP432

- The clock system includes the following external clock resources:
  - **LFXTCLK**: Low-frequency oscillator (LFXT)
    - 32-kHz or below range.
  - **HFXTCLK**: High-frequency oscillator (HFXT)
    - 1-MHz to 48-MHz range.



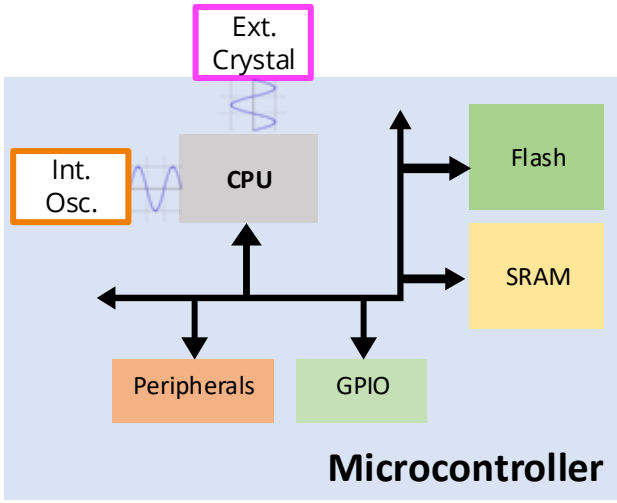
Clocks in MSP432

- The clock system includes the following internal clock resources:
  - **DCOCLK**: Internal digitally controlled oscillator (DCO)
    - Programmable frequencies (3-MHz frequency by default)
  - **VLOCLK**: Internal very-low-power low-frequency oscillator (VLO)
    - 9.4-kHz typical frequency



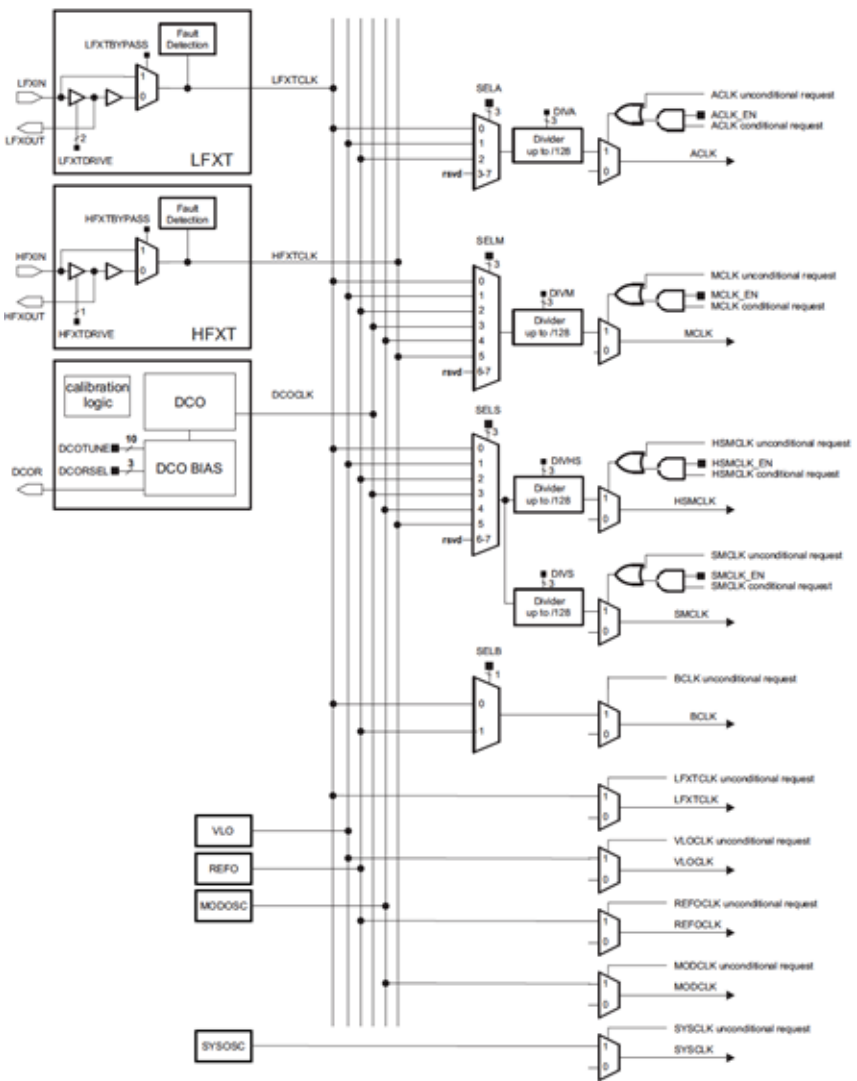
Clocks in MSP432

- The clock system includes the following internal clock resources:
  - **REFOCLK**: Internal, low-power low-frequency oscillator (REFO)
    - Selectable 32.768-kHz or 128-kHz frequencies
  - **MODCLK**: Internal low-power oscillator
    - with 25-MHz typical frequency
  - **SYSOSC**: Internal oscillator
    - with 5-MHz typical frequency



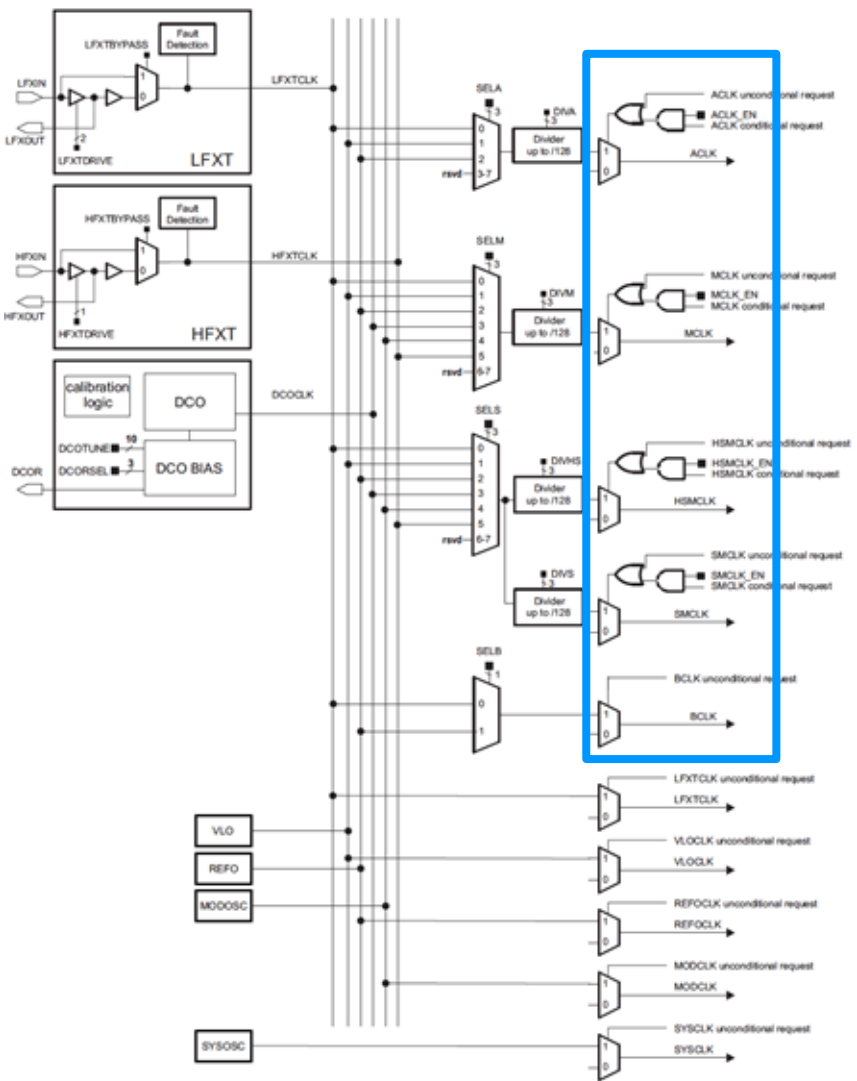
Clock System in MSP432

- Five primary system clock signals are available from the clock module:
  - **ACLK**: Auxiliary clock
  - **MCLK**: Master clock -> **To CPU**
  - **HSMCLK**: Subsystem master clock.
  - **SMCLK**: Low-speed subsystem master clock.
  - **BCLK**: Low-speed backup domain clock.



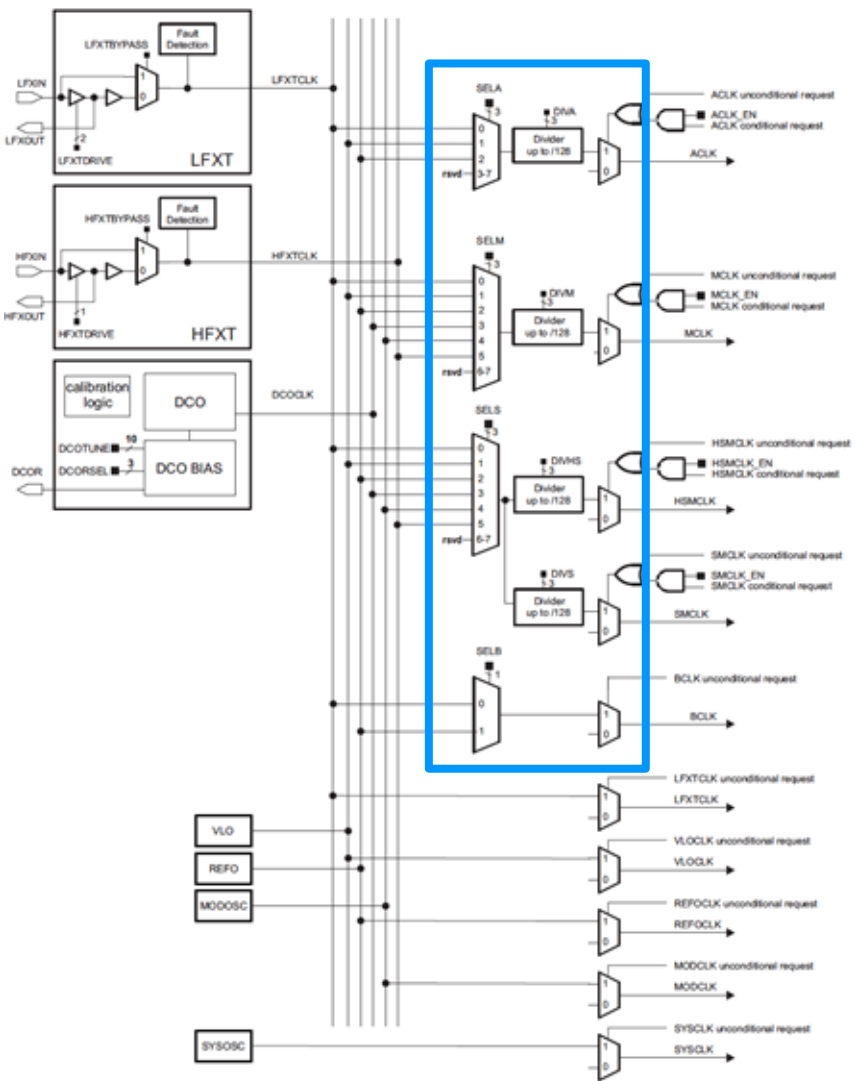
Clock System in MSP432

- Five primary system clock signals are available from the clock module:
  - **ACLK**: Auxiliary clock
  - **MCLK**: Master clock -> **To CPU**
  - **HSMCLK**: Subsystem master clock.
  - **SMCLK**: Low-speed subsystem master clock.
  - **BCLK**: Low-speed backup domain clock.



Clock System in MSP432

- The clock system can be configured or reconfigured by software at any time during program execution.
  - We have selection and divider signals.

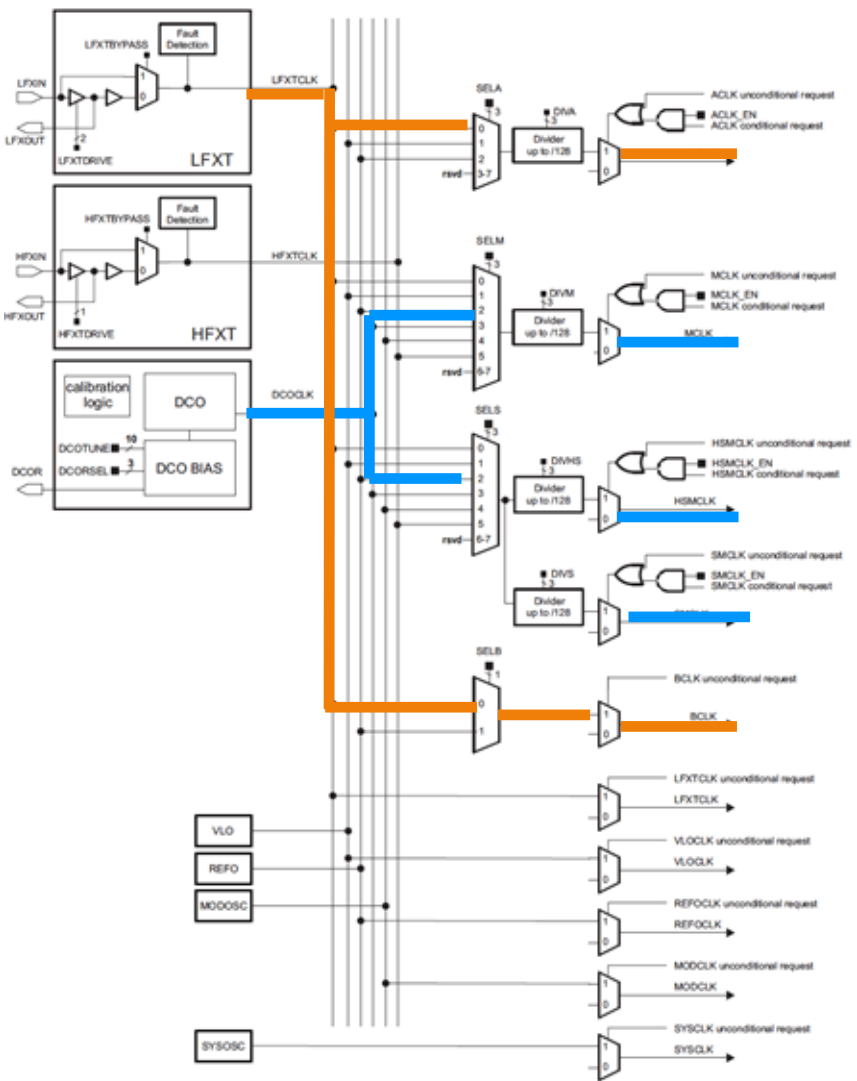


Clock System in MSP432

- MSP-EXP432P401R Default Configuration
  - LFXTCLK : 32-kHz crystal
  - HFXTCLK: 48-MHz crystal

Table 2. Default Clock Operation

Clock	Default Clock Source	Default Clock Frequency	Description
MCLK	DCO	3 MHz	Master Clock Sources CPU and peripherals
HSMCLK	DCO	3 MHz	Subsystem Master Clock Sources peripherals
SMCLK	DCO	3 MHz	Low-speed subsystem master clock Sources peripherals
ACLK	LFXT (or REFO if no crystal present)	32.768 kHz	Auxiliary clock Sources peripherals
BCLK	LFXT (or REFO if no crystal present)	32.768 kHz	Low-speed backup domain clock Sources LPM peripherals



Example Configuration

- Unlock CS module for register access
  - All clock system registers are **password protected** to prevent inadvertent access

```
CS->KEY = CS_KEY_VAL;
```

5.3.1 CSKEY Register (offset = 00h) [reset = 0000_A596h]

Clock System Key Register

Figure 5-5. CSKEY Register

31	30	29	28	27	26	25	24
Reserved							
r-0	r-0	r-0	r-0	r-0	r-0	r-0	r-0
23	22	21	20	19	18	17	16
Reserved							
r-0	r-0	r-0	r-0	r-0	r-0	r-0	r-0
15	14	13	12	11	10	9	8
CSKEY							
rw-1	rw-0	rw-1	rw-0	rw-0	rw-1	rw-0	rw-1
7	6	5	4	3	2	1	0
CSKEY							
rw-1	rw-0	rw-0	rw-1	rw-0	rw-1	rw-1	rw-0

Table 5-3. CSKEY Register Description

Bit	Field	Type	Reset	Description
31-16	Reserved	R	0h	Reserved. Always reads as 0.
15- 0	CSKEY	RW	A596h	Write CSKEY = xxxx_695Ah to unlock CS registers. All 16 LSBs need to be written together. Writing CSKEY with any other value causes CS registers to be locked and any writes to these registers are ignored, while reads are still performed. Always reads back A596h.

Example Configuration

- Set ACLK = REFO, SMCLK = MCLK = DCO

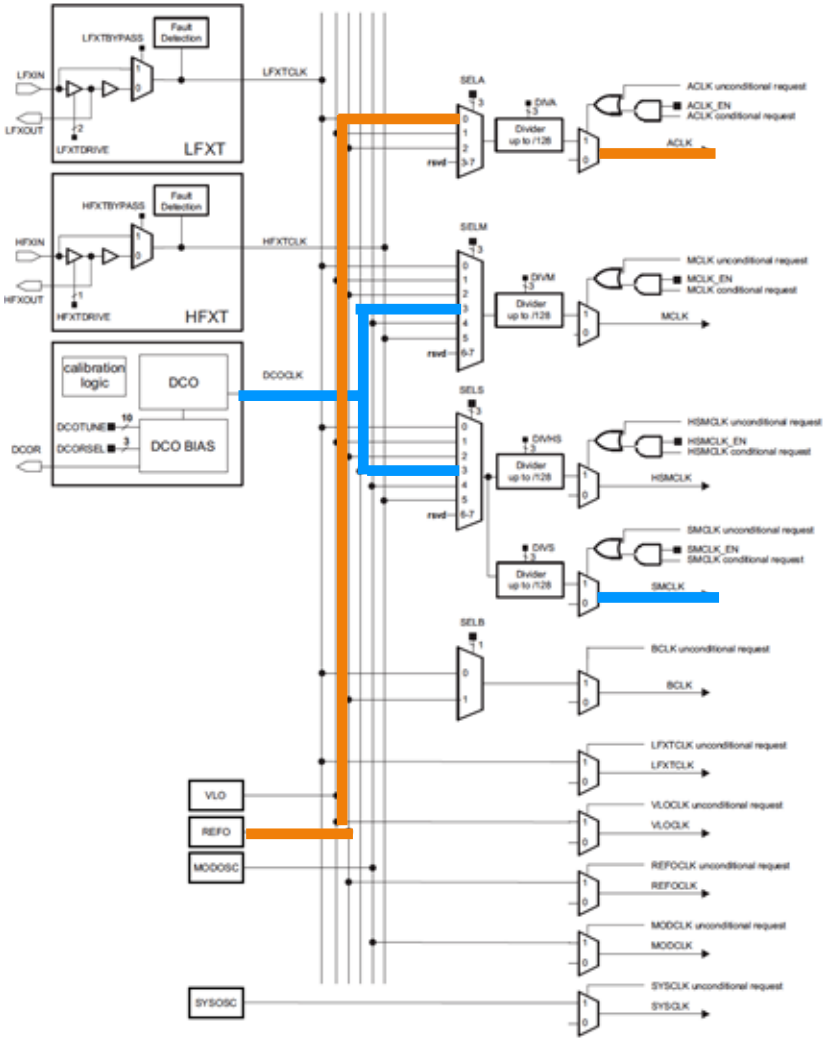


Figure 5-1. Clock System Block Diagram

Example Configuration

- Set ACLK = REFO, SMCLK = MCLK = DCO

5.3.3 CSCTL1 Register (offset = 08h) [reset = 0000_0033h]

Clock System Control 1 Register

Figure 5-7. CSCTL1 Register

31	30	29	28	27	26	25	24
Reserved	DIVS			Reserved	DIVA		
r-0	rw-0	rw-0	rw-0	r-0	rw-0	rw-0	rw-0
23	22	21	20	19	18	17	16
Reserved	DIVHS			Reserved	DIVM		
r-0	rw-0	rw-0	rw-0	r-0	rw-0	rw-0	rw-0
15	14	13	12	11	10	9	8
Reserved	Reserved		SELB	Reserved	SELA		
r-0	r-0	r-0	rw-0	r-0	rw-0	rw-0	rw-0
7	6	5	4	3	2	1	0
Reserved	SELS			Reserved	SELM		
r-0	rw-0	rw-1	rw-1	r-0	rw-0	rw-1	rw-1

Table 5-5. CSCTL1 Register Description

Bit	Field	Type	Reset	Description
31	Reserved	R	0h	Reserved. Always reads as 0.
30-28	DIVS	RW	0h	SMCLK source divider. 000b = f(SMCLK)/1 001b = f(SMCLK)/2 010b = f(SMCLK)/4 011b = f(SMCLK)/8 100b = f(SMCLK)/16 101b = f(SMCLK)/32 110b = f(SMCLK)/64 111b = f(SMCLK)/128
27	Reserved	R	0h	Reserved. Always reads as 0.
26-24	DIVA	RW	0h	ACLK source divider. 000b = f(ACLK)/1 001b = f(ACLK)/2 010b = f(ACLK)/4 011b = f(ACLK)/8 100b = f(ACLK)/16 101b = f(ACLK)/32 110b = f(ACLK)/64 111b = f(ACLK)/128
23	Reserved	R	0h	Reserved. Always reads as 0.
22-20	DIVHS	RW	0h	HSMCLK source divider. 000b = f(HSMCLK)/1 001b = f(HSMCLK)/2 010b = f(HSMCLK)/4 011b = f(HSMCLK)/8 100b = f(HSMCLK)/16 101b = f(HSMCLK)/32 110b = f(HSMCLK)/64 111b = f(HSMCLK)/128
19	Reserved	R	0h	Reserved. Always reads as 0.

Example Configuration

- Set ACLK = REFO, SMCLK = MCLK = DCO

```
CS->KEY = CS_KEY_VAL;
CS->CTL1 = CS_CTL1_SELA_2|
           CS_CTL1_SELS_3|
           CS_CTL1_SELM_3;
CS->KEY = 0;
```

Table 5-5. CSCTL1 Register Description (continued)

Bit	Field	Type	Reset	Description
18-16	DIVM	RW	0h	MCLK source divider. 000b = f(MCLK)/1 001b = f(MCLK)/2 010b = f(MCLK)/4 011b = f(MCLK)/8 100b = f(MCLK)/16 101b = f(MCLK)/32 110b = f(MCLK)/64 111b = f(MCLK)/128
15-13	Reserved	R	0h	Reserved. Always reads as 0.
12	SELB	RW	0h	Selects the BCLK source. 0b = LFXTCLK 1b = REFOCLK
11	Reserved	R	0h	Reserved. Always reads as 0.
10-8	SELA	RW	0h	Selects the ACLK source. 000b = LFXTCLK 001b = VLOCLK 010b = REFOCLK 011b-111b = Reserved for future use. Defaults to REFOCLK. Not recommended for use to ensure future compatibilities.
7	Reserved	R	0h	Reserved. Always reads as 0.
6-4	SELS	RW	3h	Selects the SMCLK and HSMCLK source. 000b = LFXTCLK 001b = VLOCLK 010b = REFOCLK 011b = DCOCLK 100b = MODOSC 101b = HFXTCLK 110b-111b = Reserved for future use. Defaults to DCOCLK. Not recommended for use to ensure future compatibilities.
3	Reserved	R	0h	Reserved. Always reads as 0.
2-0	SELM	RW	3h	Selects the MCLK source. 000b = LFXTCLK 001b = VLOCLK 010b = REFOCLK 011b = DCOCLK 100b = MODOSC 101b = HFXTCLK 110b-111b = Reserved for future use. Defaults to DCOCLK. Not recommended for use to ensure future compatibilities.

Example Configuration

- Use a divider to slow down CPU frequency.

```
CS->KEY = CS_KEY_VAL;
CS->CTL1 = CS_CTL1_SELA_2 |
           CS_CTL1_SELS_3 |
           CS_CTL1_SELM_3 |
           CS_CTL1_DIVM_7;
CS->KEY = 0;
```

Divider -> MCLK/8 = (3MHZ / 8)

Lab#1:

- Using a timer, measure the execution time of a for loop with 20000 iteration when CPU is operating at
  - 3 MhZ
  - 1 MhZ